

ระบบผู้ช่วยจัดการพื้นที่จอดรถอัตโนมัติ

Automated Parking Management Assistant System

นางสาวปวีศา	ธาระนัย	รหัสนิสิต 65050091
นายวรินทร์	จิตกลาง	รหัสนิสิต 65050095
นางสาวเหมือนฝัน	จันทสร	รหัสนิสิต 65050102

รายงานสรุปโครงงานฉบับนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต

สาขาวิชาวิศวกรรมระบบสมองกลฝังตัวและอิเล็กทรอนิกส์สื่อสาร

ภาควิชาวิศวกรรมไฟฟ้า คณะวิศวกรรมศาสตร์

มหาวิทยาลัยบูรพา

ปีการศึกษา 2568

Automated Parking Management Assistant System

Pavarisa	Tharanai	Student ID 65050091
Warinthon	Jitklang	Student ID 65050095
Mueanfan	Chanthasorn	Student ID 65050102

A PROJECT SUMMARY REPORT SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF BACHELOR OF ENGINEERING
EMBEDDED SYSTEM AND ELECTRONIC COMMUNICATION ENGINEERING
EMBEDDED SYSTEMS ENGINEERING
DEPARTMENT OF ELECTRICAL ENGINEERING
BURAPHA UNIVERSITY 2025

ชื่อโครงการ (ภาษาไทย) ระบบผู้ช่วยจัดการพื้นที่จอดรถอัตโนมัติ
(ภาษาอังกฤษ) Automated Parking Management Assistant System

โดย 1) นางสาวปวีศา ธาระนัย รหัสนิสิต 65050091
2) นายวรินทร์ จิตกลาง รหัสนิสิต 65050095
3) นางสาวเหมือนฝัน จันทศร รหัสนิสิต 65050102

อาจารย์ที่ปรึกษา ดร. สมเจต ศุภรังสรรค์

จำนวนหน้า 63 หน้า

ปีการศึกษา 2568

ภาควิชาวิศวกรรมไฟฟ้า คณะวิศวกรรมศาสตร์ มหาวิทยาลัยบูรพา
อนุมัติรายงานสรุปโครงการฉบับนี้ เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต
สาขาวิชาวิศวกรรมระบบสมองกลฝังตัวและอิเล็กทรอนิกส์สื่อสาร

.....อาจารย์ที่ปรึกษา

(ดร. สมเจต ศุภรังสรรค์)

.....หัวหน้าภาควิชาวิศวกรรมไฟฟ้า

(ดร.ณัฐพันธ์ ถนอมสัตย์)

บทคัดย่อ

โครงการระบบผู้ช่วยจัดการพื้นที่จอดรถอัตโนมัติ (Automatic Parking Management Assistant System: APMAS) พัฒนาขึ้นเพื่อแก้ไขปัญหาการบริหารจัดการลานจอดรถในสถานที่ที่มีผู้ใช้งานจำนวนมาก เช่น มหาวิทยาลัย ห้างสรรพสินค้า และอาคารสำนักงาน ซึ่งมักประสบปัญหาการค้นหาช่องจอดรถได้ยาก การจอดรถไม่เป็นระเบียบ และการจอดรถทับสิทธิ์ของผู้อื่น ระบบนี้ได้นำเทคโนโลยีด้านฮาร์ดแวร์และซอฟต์แวร์มาบูรณาการทำงานร่วมกัน โดยประกอบด้วยส่วนของประตูทางเข้า-ออก ระบบตรวจสอบสถานะช่องจอด และระบบออก QR Code สำหรับการใช้งาน ภายใน

ระบบมีการใช้กล้องสำหรับอ่านป้ายทะเบียนรถยนต์ เซนเซอร์ Ultrasonic สำหรับตรวจจับสถานะของช่องจอด ไมก์อัตโนมัติ และไฟ LED สำหรับแสดงสถานะของแต่ละช่องจอด

ระบบสามารถบันทึกและประมวลผลข้อมูลของรถยนต์ เช่น หมายเลขป้ายทะเบียน เวลาเข้า-ออก และสถานะของช่องจอด พร้อมทั้งอัปเดตข้อมูลแบบเรียลไทม์ผ่าน Firebase Realtime Database อีกทั้งยังสามารถสั่งการอุปกรณ์ฮาร์ดแวร์ให้ทำงานได้โดยอัตโนมัติ โครงการนี้พัฒนาในรูปแบบต้นแบบ (Prototype) เพื่อสาธิตการทำงานของระบบจริง โดยมีวัตถุประสงค์เพื่อช่วยลดความแออัดภายในลานจอดรถ เพิ่มความเป็นระเบียบ และยกระดับประสิทธิภาพในการบริหารจัดการพื้นที่จอดรถให้มีความสะดวก รวดเร็ว และแม่นยำยิ่งขึ้น

คำสำคัญ: ระบบจัดการลานจอดรถอัตโนมัติ, การตรวจจับป้ายทะเบียน, การประมวลผลภาพ, Raspberry Pi, Firebase Realtime Database, เซนเซอร์ Ultrasonic, QR Code

กิตติกรรมประกาศ

รายงานสรุปโครงงานฉบับนี้สำเร็จลุล่วงไปได้ด้วยความกรุณาและความช่วยเหลือเป็นอย่างดีจาก ดร. สมเจต ศุภรังสรรค์ อาจารย์ที่ปรึกษาโครงงาน ที่ได้ให้คำแนะนำ ข้อเสนอแนะ และแนวทางในการดำเนินโครงงาน ตลอดจนตรวจสอบและให้คำปรึกษาอย่างใกล้ชิดมาโดยตลอด ผู้จัดทำโครงงานขอกราบขอบพระคุณเป็นอย่างสูงไว้ ณ โอกาสนี้

ขอขอบพระคุณคณะกรรมการสอบโครงงานทุกท่าน ที่ได้กรุณาให้คำแนะนำ ข้อคิดเห็น และข้อเสนอแนะอันเป็นประโยชน์ ซึ่งช่วยให้โครงงานฉบับนี้มีความสมบูรณ์ยิ่งขึ้น รวมถึงขอขอบพระคุณคณาจารย์ทุกท่านที่ได้ถ่ายทอดความรู้และประสบการณ์อันเป็นพื้นฐานสำคัญในการจัดทำโครงงานครั้งนี้

นอกจากนี้ ผู้จัดทำขอขอบคุณครอบครัว เพื่อน และผู้ที่มีส่วนเกี่ยวข้องทุกท่าน ที่คอยให้กำลังใจ สนับสนุน และให้ความช่วยเหลือในด้านต่าง ๆ จนทำให้โครงงานฉบับนี้สำเร็จลุล่วงไปได้ด้วยดี

สารบัญ

	หน้า
บทคัดย่อ.....	I
กิตติกรรมประกาศ.....	II
สารบัญ.....	III
สารบัญรูปภาพ.....	V
สารบัญตาราง.....	VI
บทที่ 1 บทนำ.....	2
1.1 ที่มาและปัญหา.....	2
1.2 วัตถุประสงค์.....	3
1.3 ขอบเขตของโครงการ.....	3
1.4 แผนการดำเนินงาน.....	4
1.5 ประโยชน์ที่คาดว่าจะได้รับ.....	5
บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง.....	6
2.1 Raspberry Pi.....	7
2.1.1 ซีรีส์โมดูลประมวลผล.....	7
2.1.2 ส่วน GPIO.....	7
2.1.3 RPI 4 เทียบกับ RPI 5.....	8
2.2 Raspberry Pi Camera Module v3	9
2.3 ESP 32.....	10
2.4 Buzzer.....	11
2.5 Ultrasonic Sensor.....	11
2.6 Servo Motor.....	12
2.7 YOLOv8.....	12
2.8 เทคนิคเชิงวิศวกรรมศาสตร์.....	13
2.8.1 การฝึกโมเดลตรวจจับป้ายทะเบียนด้วย YOLOv8.....	13
2.8.2 กระบวนการฝึกโมเดล (Training).....	14
2.8.3 ฟังก์ชัน Loss ที่ใช้ในการฝึก.....	14
2.8.4 Metrics ที่ใช้วัดประสิทธิภาพ.....	15
2.8.5 ผลการฝึกโมเดล.....	15

สารบัญ (ต่อ)

	หน้า
2.8.6 การ Export โมเดลและการทำงานบน Edge Device.....	17
2.8.7 ระบบอ่านป้ายทะเบียนบน Raspberry Pi 4 (ระบบทางประตูเข้า).....	17
2.8.8 กลไกการโหวต (Majority Voting) เพื่อเพิ่มความแม่นยำ.....	20
2.8.9 การตรวจสอบป้ายทะเบียน (Plate Whitelist).....	21
2.8.10 การเชื่อมต่อระบบและการเปิดไม้กั้น.....	21
2.9 จุดเด่นของระบบที่ต่างจากงานทั่วไป.....	22
2.10 งานวิจัยที่เกี่ยวข้อง.....	22
บทที่ 3 วิธีดำเนินงานโครงการ.....	24
3.1 ภาพรวมของโครงการ	24
3.2. โครงสร้างระบบซอฟต์แวร์โดยรวม.....	25
3.3. โครงสร้างข้อมูลใน Firebase.....	26
3.4. ลำดับการทำงานของระบบ.....	28
บทที่ 4 ผลการทดสอบ.....	44
4.1 การทดสอบความแม่นยำในการตรวจจับป้ายทะเบียน (ทางเข้า).....	44
4.2 การทดสอบความแม่นยำในการจับคู่ป้ายทะเบียนระหว่างทางเข้ากับลานจอดรถ.....	45
4.3 การทดสอบการตัดสินใจเลือกช่องจอดรถ.....	46
4.4 การทดสอบเซนเซอร์ Ultrasonic ตรวจจับสถานะช่องจอด.....	48
4.5 การทดสอบ Buzzer เมื่อป้ายทะเบียน RPI 5 ไม่ตรงกับ Ticket ID.....	49
สรุปผลการทดสอบทั้งหมด	51
บทที่ 5 สรุปผลการทดลองและข้อเสนอแนะ	52
5.1 สรุปผลการทดลอง	52
5.2 ปัญหาที่พบ	52
5.3 ข้อเสนอแนะ	52
เอกสารอ้างอิง.....	53

สารบัญรูปภาพ

รูปที่	หน้า
2.1 Raspberry Pi 5 (ซ้าย) และ Raspberry Pi 4 (ขวา).....	6
2.2 ตำแหน่งของขา GPIO.....	8
2.3 RPI Camera Module v2 (ซ้าย) และ RPI Camera Module v3 (ขวา).....	9
2.4 ESP 32 38pin TYPE C.....	10
2.5 Passive Buzzer.....	11
2.6 Ultrasonic Sensor.....	11
2.7 Servo Motor.....	12
2.8 ภาพถ่ายป้ายทะเบียนรถยนต์ไทย จัดเก็บไว้ใน Google Drive.....	13
2.9 ตัวอย่างรูปภาพที่ใช้ในการฝึกโมเดล.....	13
2.10 ไฟล์ Label (.txt).....	14
2.11 คำสั่งที่ใช้ฝึก.....	14
2.12 กราฟแสดงผล Training Metrics ตลอด 250 Epoch (train/val loss, Precision, Recall, mAP).....	15
2.12 Confusion Matrix แสดงผลการ Validate โมเดล YOLOv8 บน Validation Set.....	16
2.13 การ Export โมเดลและการใช้งานบน Edge Device.....	17
2.14 define ของ Letterbox.....	18
2.15 define ของ Non-Maximum Suppression (NMS).....	19
2.16 define ของ Preprocessing.....	20
2.17 ผลการโหวต.....	21
3.1 APMAS Overview Diagram.....	25
3.2 การทำงานของ RPI 5 ขณะเริ่มระบบ.....	28
3.3 การทำงานของระบบตรวจจับรถและอ่านป้ายทะเบียน.....	29

สารบัญรูปภาพ(ต่อ)

รูปที่	หน้า
3.4 การทำงานของระบบจัดช่องจอดและอัปเดตสถานะ.....	31
3.5 การทำงานของระบบ Server สร้าง QR Code.....	32
3.6 การทำงานของระบบเปิดไม้กั้นและแสดง Slip.....	33
3.7 การทำงานของระบบการตรวจสอบการจอดในช่อง.....	34
3.8 การทำงานของระบบรถออกจากช่องจอด และระบบ Reset.....	36
3.9 การทำงานของระบบผู้ใช้ชำระเงินผ่าน QR Code.....	36
3.10 ต่อกล้องเข้ากับบอร์ด RPI 5 และ Pin ทั้งหมด.....	37
3.11 การต่อกล้องเข้ากับบอร์ด RPI 4 และ Pin ทั้งหมด.....	40
3.12 Pin ทั้งหมด ของ ESP 32.....	42

สารบัญตาราง

ตารางที่	หน้า
2.1 RPI 4 เทียบกับ RPI 5.....	8
2.2 ค่า Score ของการตั้งค่ากล้อง IMX219 ในรูปแบบต่าง ๆ (ค่าสูง = คุณภาพดีกว่า).....	18
2.3 ค่าความมั่นใจของแต่ละอักขระจากการอ่านป้ายทะเบียน รถ2015.....	20
2.4 ผลการอ่านป้ายทะเบียนใน 3 รอบและค่าความแม่นยำของแต่ละรอบ.....	21
3.1 องค์ประกอบของระบบซอฟต์แวร์ทั้งหมด.....	26
3.2 โฟลเดอร์ tickets เก็บข้อมูลรถแต่ละคันที่เข้าใช้บริการ.....	26
3.3 สถานะ (status) ของ ticket เปลี่ยนแปลงตาม.....	27
3.4 โฟลเดอร์ slots เก็บสถานะช่องจอด.....	27
3.5 กระบวนการนำรถออกจากลานจอดรถ.....	36
3.6 การต่อ Ultrasonic Sensor 6 ตัว.....	38
3.7 การต่อ LED สีเขียว 6 ตัว.....	39
3.8 การของ Buzzer 6 ตัว.....	40
3.9 การต่อ Servo Motor (ไม้กั้นทางเข้า) PRI 4	41
3.10 การต่อ Servo Motor (ไม้กั้นทางเข้า) ESP 32.....	43
4.1 ผลการทดสอบอ่านป้ายทะเบียนทางเข้า.....	44
4.2 ผลการจับคู่ป้ายทะเบียนระหว่างทางเข้ากับลานจอดรถ.....	45
4.3 ผลการทดสอบการเลือกช่องจอด.....	47
4.4 ผลการทดสอบ Ultrasonic ตรวจจับสถานะช่องจอด.....	48
4.5 ผลการทดสอบ Buzzer เมื่อป้ายทะเบียนไม่ตรง.....	49

บทที่ 1

บทนำ

1.1 ที่มาและปัญหา

ปัญหาการจัดการพื้นที่จอดรถในสถานที่ที่มีผู้ใช้งานจำนวนมาก เช่น มหาวิทยาลัย ห้างสรรพสินค้า และอาคารสำนักงาน มักเกิดจากการหาที่จอดรถยาก การจอดผิดตำแหน่ง การจอดทับสิทธิ์ รวมถึงการใช้เวลาในการวนหาช่องจอดว่าง ส่งผลให้เกิดความล่าช้า ความแออัด และความไม่เป็นระเบียบ นอกจากนี้ ระบบจัดการลานจอดรถแบบเดิมยังขาดความแม่นยำในตรวจสอบข้อมูล เช่น ป้ายทะเบียน เวลาจอด และการชำระเงิน ทำให้ผู้ดูแลไม่สามารถบริหารจัดการได้อย่างเต็มประสิทธิภาพ โครงการงาน “ระบบผู้ช่วยจัดการพื้นที่จอดรถอัตโนมัติ (Automated Parking Management Assistant System : APMAS)” จึงถูกพัฒนาขึ้นเพื่อตอบโจทย์ปัญหาเหล่านี้ โดยใช้เทคโนโลยีร่วมกันทั้งด้านฮาร์ดแวร์และซอฟต์แวร์

APMAS ออกแบบให้มียูนิทประกอบทั้งส่วนประตูทางเข้า-ออก ส่วนลานจอด และระบบออกบัตรพร้อม QR Code โดยมีเซนเซอร์ตรวจจับรถ กล้องอ่านป้ายทะเบียน ไม้กั้นอัตโนมัติ ไฟ LED แสดงสถานะช่องจอด และระบบตรวจจับการเคลื่อนไหวของรถจาก Ultrasonic มาทำงานร่วมกัน ซอฟต์แวร์จะประมวลผลข้อมูล เช่น ป้ายทะเบียน เวลาเข้า-ออก ตำแหน่งช่องว่าง และส่งไปยังส่วนฮาร์ดแวร์เพื่อควบคุมการทำงาน เช่น การออกบัตร การเปิด-ปิดไม้กั้น และการแจ้งเตือนเมื่อพบความผิดปกติ เช่น รถผิดคันหรือยังไม่ชำระเงินก่อนออก

ทั้งนี้ เพื่อให้ระบบมีความสมจริงแต่เหมาะสมกับระยะเวลาทำโครงการ ส่วนฮาร์ดแวร์จะทำในลักษณะจำลองการทำงาน (Prototype) ไม่ใช่ขนาดจริง แต่สามารถแสดงการทำงานได้ ระบบนี้จึงมีความสำคัญในการยกระดับการจัดการที่จอดรถให้ทันสมัย เป็นระเบียบ ลดความแออัด และเพิ่มความสะดวกแก่ผู้ใช้งานและผู้ให้บริการ

1.2 วัตถุประสงค์

- 1.2.1 เพื่อพัฒนาระบบที่ช่วยให้ผู้ใช้งานสามารถจองพื้นที่จอดรถล่วงหน้าได้อย่างสะดวกและเป็นระบบ
- 1.2.2 เพื่อพัฒนาระบบตรวจสอบข้อมูลรถยนต์อัตโนมัติ ได้แก่ การอ่านป้ายทะเบียน การบันทึกเวลาเข้า-ออก และการตรวจสอบสิทธิ์การจอดรถ
- 1.2.3 เพื่อเพิ่มประสิทธิภาพในการบริหารจัดการช่องจอดรถ ลดปัญหาการจอดผิดตำแหน่ง การจอดทับสิทธิ์ และความสับสนในการใช้งานพื้นที่จอดรถ
- 1.2.4 เพื่อพัฒนาระบบตรวจจับและแสดงสถานะช่องจอดรถโดยใช้เซนเซอร์และไฟแสดงผลเพื่อลดระยะเวลาในการค้นหาช่องจอดและลดความแออัดภายในลานจอดรถ
- 1.2.5 เพื่อ Prototype ที่สามารถสาธิตการทำงานจริงของระบบจัดการพื้นที่จอดรถอัตโนมัติและเป็นแนวทางในการพัฒนาไปสู่การใช้งานจริงในอนาคต

1.3 ขอบเขตของโครงการ

1.3.1 ขอบเขตด้านฮาร์ดแวร์ (Hardware Scope – Prototype)

- สร้างแบบจำลอง (ไม่ใช่ขนาดจริง) ของส่วนประตูทางเข้า-ออก ประกอบด้วย ไมก์กัน ช่องจอด 6 ช่อง เซนเซอร์ตรวจจับรถ และกล้องอ่านป้ายทะเบียน
- ระบบจะออก QR Code ตามข้อมูลที่ได้รับจากระบบซอฟต์แวร์
- ต้นแบบลานจอดรถจำลองที่ประกอบด้วย LED แสดงสถานะช่องว่าง-ไม่ว่าง เซนเซอร์ Ultrasonic สำหรับตรวจจับรถ และลำโพงสำหรับแจ้งเตือน
- ระบบจำลองการเปิด-ปิดไมก์กันตามการตรวจสอบป้ายทะเบียนและสถานะการชำระเงิน

1.3.2 ขอบเขตด้านระบบการทำงาน (System Process Scope)

- ระบบตรวจจับและยืนยันป้ายทะเบียนรถเมื่อเข้า-ออกพื้นที่จอด
- ระบบคำนวณเวลาและค่าบริการจอดรถโดยอัตโนมัติผ่านข้อมูลที่อ่านจาก QR Code

1.5 ประโยชน์ที่คาดว่าจะได้รับ

1.5.1 ช่วยแก้ปัญหาการหาที่จอดรถลำบากและลดการหมุนหาที่จอดโดยไม่จำเป็น ระบบสามารถตรวจจับจำนวนที่จอดว่างแบบเรียลไทม์และแสดงผลให้ผู้ขับรถทราบทันที ทำให้ผู้ขับไม่ต้องขับวนหาที่จอดนาน ช่วยลดความแออัดและความเครียดในการหาพื้นที่จอดรถ

1.5.2 ลดปัญหาการจอดไม่เป็นระเบียบและลดความผิดพลาดจากมนุษย์ด้วยเซนเซอร์หรือกล้องอัตโนมัติ ระบบสามารถตรวจสอบการเข้า-ออกของรถได้อย่างแม่นยำ ทำให้การจัดการพื้นที่จอดรถมีความเป็นระบบมากขึ้น ลดโอกาสการจอดกีดขวางหรือจอดผิดกฎระเบียบ

1.5.3 เพิ่มประสิทธิภาพการบริหารจัดการพื้นที่จอดรถ ระบบช่วยให้ผู้ดูแลสามารถตรวจสอบข้อมูลที่จอดว่าง เต็ม และสถิติการใช้งานได้ทันที ทำให้ง่ายต่อการวางแผน ปรับปรุง หรือขยายพื้นที่จอดรถในอนาคต

1.5.4 ลดปัญหาการจราจรติดขัดในบริเวณที่จอดรถ เมื่อผู้ขับรู้ตำแหน่งที่จอดล่วงหน้า ทำให้ลดการชะลอรถหรือหยุดรอแบบไม่จำเป็น ลดความหนาแน่นในพื้นที่อาคารหรือสถานที่ที่มีรถจำนวนมาก

1.5.5 เพิ่มความทันสมัยและความปลอดภัยในพื้นที่จอดรถ ระบบสามารถต่อยอดด้วยการติดตั้งกล้องหรือระบบตรวจจับเพิ่มเติม เพื่อเพิ่มความปลอดภัยและทำให้บริเวณจอดรถมีความเป็นอัจฉริยะมากขึ้น

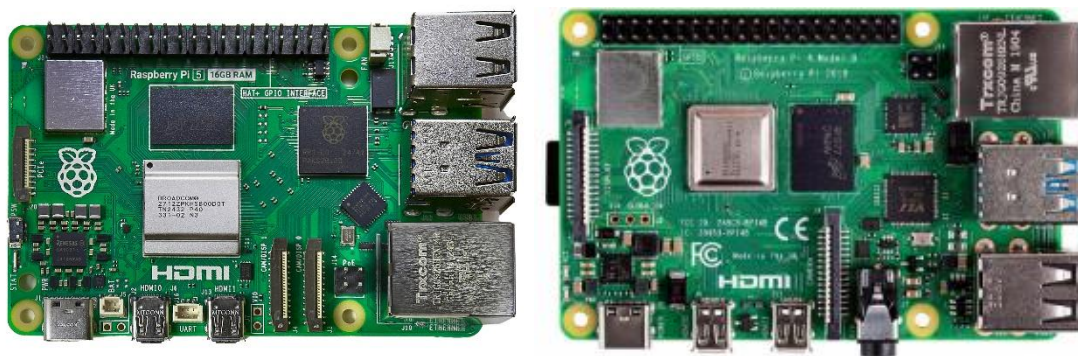
บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

APMAS มีทฤษฎีที่จำเป็นเพื่อใช้ในการอ้างอิงหรือใช้เป็นแนวทางในการออกแบบโครงงานที่ผู้จัดทำได้ทำการรวบรวมทฤษฎีและงานวิจัยที่เกี่ยวข้องดังนี้

2.1 Raspberry Pi

Raspberry Pi (RPI) ก็เหมือนคอมพิวเตอร์ตัวเล็ก ๆ ที่รวมทุกอย่างไว้บนบอร์ดเดียว พัฒนาขึ้นที่ประเทศอังกฤษโดยมูลนิธิ RPI ร่วมกับบริษัท Broadcom ตอนแรกตั้งใจทำมาเพื่อใช้สอนวิชาคอมพิวเตอร์ในโรงเรียน เพราะราคาถูก ขนาดเล็ก และใช้งานได้หลากหลาย แต่ปัจจุบันมันถูกนำไปใช้กว้างมาก ทั้งงานระบบอัตโนมัติ หุ่นยนต์ บ้านอัจฉริยะ IoT และงานอิเล็กทรอนิกส์ต่าง ๆ ตัวเครื่องใช้ชิปแบบ SoC มีพอร์ตสำคัญครบ เช่น HDMI, USB, Wi-Fi, GPIO และใส่ RAM ได้สูงสุดถึง 16GB โดยส่วนใหญ่เก็บข้อมูลผ่านการ์ด microSD ซึ่งทำให้ใช้งานได้เหมือนคอมพิวเตอร์ขนาดย่อมเลยทีเดียว และด้วยความคุ้มค่าและความยืดหยุ่น ทำให้ RPI ได้รับความนิยมทั่วโลก จนในปี 2015 มียอดขายแซง ZX Spectrum และภายในเดือนมีนาคม ปี 2025 มียอดขายรวมมากกว่า 68 ล้านเครื่องแล้ว



รูปที่ 2.1 RPi 5 (ซ้าย) และ RPi 4 (ขวา)

(ที่มา: https://en.wikipedia.org/wiki/Raspberry_Pi)

2.1.1 ซีรีส์โมดูลประมวลผล

Computer Module หรือ CM คือบอร์ด RPI แบบพิเศษที่ออกแบบมาให้ใช้ในงานอุตสาหกรรมและระบบฝังตัว โดยเอาส่วนฮาร์ดแวร์หลักของ RPI มาใส่ไว้ในบอร์ดขนาดเล็กเพื่อให้ต่อเข้ากับบอร์ดหลัก (Carrier Board) ได้ง่ายกว่า ไม่ได้เน้นใช้งานพอร์ต GPIO แบบบอร์ดทั่วไป โมดูลนี้มี 2 รูปแบบหลัก คือแบบ SO-DIMM ที่หน้าตาคล้ายแรมคอม (แต่ใช้แทนกันไม่ได้) และแบบขั้วต่อความหนาแน่นสูง 100 พินที่รองรับการเชื่อมต่ออุปกรณ์ได้มากขึ้น

รุ่นของ CM ก็พัฒนาตาม RPI รุ่นหลักเรื่อย ๆ เริ่มจาก CM1 (2014) ที่ใช้ CPU ARM11 แขนเดียว RAM 512MB และ eMMC 4GB ต่อมา CM3 (2017) และ CM3+ (2019) อัปเกรดเป็น CPU Quad-core 64 บิต พร้อมเพิ่มตัวเลือกความจุ eMMC หลายขนาด หลังจากนั้น CM4 (2020) ที่อิงจาก RPI 4 มีตัวเลือก RAM หลายระดับ เพิ่ม Wi-Fi/Bluetooth และเปลี่ยนมาใช้ขั้วต่อความหนาแน่นสูง ส่วนรุ่นล่าสุดคือ CM5 (2024) ที่พัฒนาจาก RPI 5 ใช้ CPU Cortex-A76 ประสิทธิภาพสูงขึ้น รองรับ RAM สูงสุด 16GB และ eMMC สูงสุด 64GB เหมาะกับงานที่ต้องการความเสถียรและการออกแบบฮาร์ดแวร์เฉพาะทางมาก ๆ

2.1.2 ส่วน GPIO

GPIO ของ RPI ก็คือชุดขาเชื่อมต่อบนบอร์ดที่เอาไว้คุยกับอุปกรณ์ภายนอก ส่วนใหญ่จะมีทั้งหมด 40 พิน (เรียกว่า GPIO Header หรือ J8 Header) โดยจัดเรียงเหมือนกันหลายรุ่น ทำให้ใช้งานหรือย้ายวงจรได้ง่าย ภายในชุดพินนี้จะมีทั้งขาจ่ายไฟ 3.3V และ 5V รวมถึงขาสื่อสารต่าง ๆ เช่น UART, SPI, I²C, I²S และ PCM สำหรับต่อเซนเซอร์หรืออุปกรณ์อื่น ๆ ได้โดยตรง

การทำงานของ GPIO แบ่งง่าย ๆ เป็น 2 โหมด คือ Input ใช้รับสัญญาณไฟฟ้าเพื่ออ่านค่าจากอุปกรณ์ภายนอก และ Output ใช้ส่งสัญญาณออกไปเพื่อควบคุมอุปกรณ์ เช่น LED หรือรีเลย์ ส่วนความแตกต่างของแต่ละรุ่นคือ RPI รุ่นแรกมีแค่ 26 พิน, รุ่น Pi Zero บางรุ่นต้องบัดกรีขาเพิ่มเอง, RPI Pico จะมีรูปแบบขาเฉพาะของมัน และรุ่น CM จะไม่มีหัว GPIO บนบอร์ด Carrier Board

GPIO Numbers

Raspberry Pi B
Rev 1 P1 GPIO Header

Pin No.		
3.3V	1	2 5V
GPIO0	3	4 5V
GPIO1	5	6 GND
GPIO4	7	8 GPIO14
GND	9	10 GPIO15
GPIO17	11	12 GPIO18
GPIO21	13	14 GND
GPIO22	15	16 GPIO23
3.3V	17	18 GPIO24
GPIO10	19	20 GND
GPIO9	21	22 GPIO25
GPIO11	23	24 GPIO8
GND	25	26 GPIO7

Raspberry Pi A/B
Rev 2 P1 GPIO Header

Pin No.		
3.3V	1	2 5V
GPIO2	3	4 5V
GPIO3	5	6 GND
GPIO4	7	8 GPIO14
GND	9	10 GPIO15
GPIO17	11	12 GPIO18
GPIO27	13	14 GND
GPIO22	15	16 GPIO23
3.3V	17	18 GPIO24
GPIO10	19	20 GND
GPIO9	21	22 GPIO25
GPIO11	23	24 GPIO8
GND	25	26 GPIO7

Raspberry Pi B+
B+ J8 GPIO Header

Pin No.		
3.3V	1	2 5V
GPIO2	3	4 5V
GPIO3	5	6 GND
GPIO4	7	8 GPIO14
GND	9	10 GPIO15
GPIO17	11	12 GPIO18
GPIO27	13	14 GND
GPIO22	15	16 GPIO23
3.3V	17	18 GPIO24
GPIO10	19	20 GND
GPIO9	21	22 GPIO25
GPIO11	23	24 GPIO8
GND	25	26 GPIO7
DNC	27	28 DNC
GPIO5	29	30 GND
GPIO6	31	32 GPIO12
GPIO13	33	34 GND
GPIO19	35	36 GPIO16
GPIO26	37	38 GPIO20
GND	39	40 GPIO21

Key	
Power +	UART
GND	SPI
I ² C	GPIO

รูปที่ 2.2 ตำแหน่งของขา GPIO

(ที่มา: <http://www.arduino-makerzone.com/article/57/raspberry-pi-tutorial-ep1-raspberry-pi-คืออะไร>)

2.1.3 RPI 4 เทียบกับ RPI 5

RPI 5 นั้นแรงกว่า RPI 4 ในเกือบทุกด้าน ไม่ว่าจะเป็น CPU ที่เร็วขึ้น เน็ตเวิร์กดีขึ้น และภาพ HDMI ชัดขึ้น แต่ก็แลกมาด้วยราคาที่แพงขึ้น 550 บาท และ RAM สูงสุดน้อยกว่า ดังตารางที่ 2.1 ตารางที่ 2.1 RPI 4 เทียบกับ RPI 5

Feature	RPi 4	Raspberry Pi 5
CPU	Quad – Core Cortex – A72, 1.5GHz	Quad – Core Cortex – A76, 2.4GHz
RAM Options	1GB, 2GB, 4GB, 8GB	1GB, 2GB, 4GB
Storage	MicroSD Card	MicroSD Card
USB Ports	2x USB 3.0, 2x USB 2.0	2x USB 3.0, 2x USB 2.0
Networking	Gigabit Ethernet, Wi-Fi 5	2.5 Gbps Ethernet, Wi-Fi 6
Video Output	2x Micro-HDMI	2x Micro-HDMI (4K@60Hz)

ตารางที่ 2.1 (ต่อ) RPi 4 เทียบกับ RPi 5

Feature	RPi 4	Raspberry Pi 5
Power Supply	USB-C, 5V/3A	USB-C, 5V/5A
Price	1,200 บาท	1,750 บาท

(ที่มา : <https://www.viam.com/post/your-ultimate-guide-to-raspberry-pi-5-now-fully-supported-on-viam/>)

2.2 Raspberry Pi Camera Module v3

RPi Camera Module V2 ใช้เซ็นเซอร์ Sony IMX219 ความละเอียด 8MP พร้อมเลนส์โฟกัสคงที่มีขนาดเล็กและน้ำหนักเบา เชื่อมต่อผ่านพอร์ต CSI ด้วยสายริบบิ้น เหมาะกับงานที่ต้องการอุปกรณ์ขนาดเล็กกะทัดรัด สามารถถ่ายภาพนิ่งได้สูงสุด 3280×2464 พิกเซล และรองรับวิดีโอ 1080p30, 720p60 และ 640×480p90 โดยซอฟต์แวร์รองรับในระบบปฏิบัติการ RPi OS ทำให้ถูกนำไปใช้บ่อยในงานกล้องวงจรปิด ตรวจสอบการเคลื่อนไหว หรือการถ่ายภาพแบบ timelapse

ส่วนกล้อง RPi Camera Module V3 มีทั้งหมด 4 รุ่น คือ รุ่นมาตรฐาน รุ่นมุมกว้าง รุ่น NoIR และรุ่น NoIR มุมกว้าง โดยใช้เซ็นเซอร์ Sony IMX708 ความละเอียด 12MP แบบ Rolling Shutter เหมือนกันทั้งหมด พร้อมระบบโฟกัสอัตโนมัติที่ทำงานร่วมกันระหว่าง PDAF และ CDAF เพื่อให้โฟกัสได้แม่นยำ รุ่นปกติและ NoIR มีมุมมองภาพประมาณ 66° ส่วนรุ่นมุมกว้างจะได้ประมาณ 102° และมีระยะโฟกัสต่ำสุดต่างกันคือ 5 ซม. สำหรับรุ่นปกติ และ 10 ซม. สำหรับรุ่นมุมกว้าง นอกจากนี้ยังรองรับภาพนิ่งสูงสุด 4608×2592 พิกเซล วิดีโอ 1080p50 มีโหมด HDR ใช้งานกับ RPi ได้เกือบทุกรุ่น (ยกเว้น Pi 400) และรองรับซอฟต์แวร์ใหม่อย่าง Libcamera กับ Picamera2 โดยจะยังคงผลิตต่ออย่างน้อยถึงปี 2030



รูปที่ 2.3 RPi Camera Module v2 (ซ้าย) และ RPi Camera Module v3 (ขวา)

(ที่มา : <https://th.cytron.io/c-raspberry-pi/c-raspberry-pi-camera/p-raspberry-pi-8mp-camera-module-v2> และ <https://th.cytron.io/c-raspberry-pi/c-raspberry-pi-camera/p-raspberry-pi-camera-module-3-12mp-with-auto-focus-lens>)

2.3 ESP 32

ESP32 เป็นไมโครคอนโทรลเลอร์ในกลุ่มที่มีต้นทุนต่ำและประหยัดพลังงาน โดยมีจุดเด่นคือความสามารถด้านการสื่อสารแบบไร้สายทั้ง Wi-Fi และ Bluetooth ไร้สายในชิปเดียว ชิป ESP32 มีตัวเลือกด้านหน่วยประมวลผลที่หลากหลาย เช่น ไมโครโปรเซสเซอร์ Tensilica Xtensa LX6 ซึ่งมีทั้งแบบแกนเดี่ยว (Single-Core) และแบบสองแกน (Dual-Core) รวมถึงโปรเซสเซอร์ Xtensa LX7 แบบ dual-core และไมโครโปรเซสเซอร์สถาปัตยกรรม RISC-V แบบ Single-core

นอกจากนี้ ESP32 ยังรวมองค์ประกอบที่สำคัญสำหรับการสื่อสารข้อมูลแบบไร้สายไว้ในตัว เช่น สวิตช์เสาอากาศในตัว (Antenna switch), RF balun, เครื่องขยายสัญญาณกำลัง (Power amplifier), ตัวรับสัญญาณรบกวนต่ำ (Low-noise amplifier), ตัวกรองสัญญาณ และโมดูลจัดการพลังงาน

โดยทั่วไป ESP32 มักถูกนำไปใช้งานโดยการติดตั้งลงบนแผงวงจรพิมพ์ (PCB) เฉพาะสำหรับอุปกรณ์ หรืออยู่ในรูปแบบชุดพัฒนา (Development board) ที่มีพิน GPIO และตัวเชื่อมต่อหลากหลายรูปแบบ ซึ่งมีการกำหนดค่าที่แตกต่างกันไปตามรุ่นและผู้ผลิต ทั้งนี้ ESP32 ได้รับการออกแบบโดยบริษัท Espressif Systems และผลิตโดยใช้กระบวนการผลิตขนาด 40 นาโนเมตร โดยถือเป็นรุ่นพัฒนาต่อจากไมโครคอนโทรลเลอร์ ESP8266

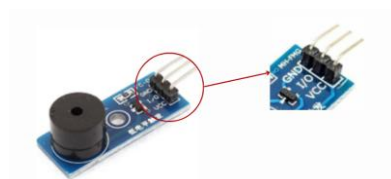


รูปที่ 2.4 ESP 32 38 Pin TYPE C

(ที่มา: <https://www.ortech-online.com/product/1886/cb001-บอร์ด-esp32-38pin-type-c-esp32s-chip-cp2102-wifi-bluetooth-board-esp-wroom-32-nodemcu>)

2.4 Buzzer

Buzzer เป็นอุปกรณ์กำเนิดเสียงที่มีทั้งแบบแม่เหล็กและแบบเพียโซ โดยภายในมีวงจรกำเนิดความถี่ (Oscillator) ในตัว สามารถทำงานได้ด้วยแรงดันไฟฟ้าประมาณ 3.3–5 โวลต์ ทำหน้าที่สร้างเสียงเพื่อใช้เป็นสัญญาณเตือนหรือแสดงสถานะในรูปแบบต่าง ๆ โดยมักถูกนำไปใช้งานในอุปกรณ์อิเล็กทรอนิกส์ เช่น เสียง “บีบ” จากคอมพิวเตอร์ที่ใช้แจ้งสถานะการทำงานหรือข้อผิดพลาดของระบบ รวมถึงสามารถประยุกต์ใช้ในระบบแจ้งเตือนอื่น ๆ ได้อย่างหลากหลาย



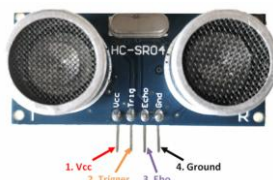
รูปที่ 2.5 Passive Buzzer

(ที่มา: <https://www.ortech-online.com/product/1886/cb001-บอร์ด-esp32-38pin-type-c-esp32s-chip-cp2102-wifi-bluetooth-board-esp-wroom-32-nodemcu>)

2.5 Ultrasonic Sensor

เซนเซอร์อัลตราโซนิก (Ultrasonic Sensor) เป็นอุปกรณ์ที่ใช้สำหรับตรวจจับและวัดระยะของวัตถุ โดยอาศัยหลักการสะท้อนของคลื่นเสียงความถี่สูง ซึ่งจะส่งคลื่นเสียงออกไปและวัดเวลาที่คลื่นสะท้อนกลับมา เพื่อนำมาคำนวณหาระยะทางจากความสัมพันธ์ระหว่างระยะทางและเวลา กลไกดังกล่าวทำให้สามารถนำไปประยุกต์ใช้งานได้หลากหลาย เช่น การวัดระดับน้ำ การตรวจจับชิ้นงาน และการวัดความหนาของวัตถุ

Ultrasonic Sensor ใช้คลื่นเสียงที่มีความถี่สูงกว่า 20 กิโลเฮิร์ตซ์ (kHz) ซึ่งอยู่นอกช่วงการได้ยินของมนุษย์ โดยคลื่นสามารถเคลื่อนที่ผ่านตัวกลางได้หลายประเภท เช่น อากาศ ก๊าซ ของเหลว และของแข็ง (ยกเว้นในสภาวะสุญญากาศ) จึงเหมาะสำหรับการใช้งานในสภาพแวดล้อมที่หลากหลาย หลักการทำงานคือการส่งคลื่นเสียงออกไปแล้วตรวจวัดเวลาที่คลื่นสะท้อนกลับมาเพื่อคำนวณระยะทาง ซึ่งมีความแม่นยำค่อนข้างสูงเนื่องจากคลื่นมีทิศทางเคลื่อนที่ที่ชัดเจน นอกจากนี้ยังถูกนำไปใช้งานในหลายด้าน เช่น การวัดระยะ การวัดความลึก การทำความสะอาดด้วยคลื่นเสียง และการทำงานทางการแพทย์ โดยความถี่ที่ใช้งานจะแตกต่างกันไปตามลักษณะงาน เช่น งานในอากาศมักใช้ความถี่ไม่เกิน 50 KHz ส่วนงานทางการแพทย์จะใช้ความถี่ประมาณ 1–10 MHz ทั้งนี้ ในโครงงานนี้เลือกใช้เซนเซอร์ที่มีความถี่ประมาณ 40 KHz



รูปที่ 2.6 Ultrasonic Sensor

(ที่มา: <https://components101.com/sensors/ultrasonic-sensor-working-pinout-datasheet>)

2.6 Servo Motor

คือมอเตอร์ที่ถูกออกแบบมาให้ควบคุมการทำงานได้อย่างแม่นยำ ทั้งตำแหน่ง ความเร็ว และ แรงบิด โดยทำงานร่วมกับระบบไฟฟ้าและเครื่องกล พร้อมมีระบบป้อนกลับ (Feedback) คอย ตรวจสอบผลลัพธ์แล้วปรับการทำงานให้ตรงกับค่าที่ตั้งอยู่ตลอดเวลา จึงทำให้สามารถเคลื่อนที่ได้อย่างรวดเร็ว แม่นยำ และมีประสิทธิภาพสูง ปัจจุบันมีทั้งแบบกระแสตรง (DC) และกระแสสลับ (AC) และถูกนำไปใช้อย่างแพร่หลายในงานที่ต้องการความละเอียดและความเที่ยงตรงสูง เช่น ระบบอัตโนมัติและ เครื่องจักรต่าง ๆ



รูปที่ 2.7 Servo Motor

(ที่มา: <https://shopee.co.th/SG90-Servo-Motor-0-180-องศาสำหรับ-Arduino-i.56846818.6435198442>)

2.7 YOLOv8

YOLO (You Only Look Once) คือเทคโนโลยีสำหรับตรวจจับวัตถุในภาพที่ได้รับความนิยมมาก เพราะสามารถวิเคราะห์ภาพเพียงครั้งเดียวแล้วบอกได้ทันทีว่ามีวัตถุอะไรอยู่ตรงไหน ทำให้ทั้งเร็วและแม่นยำ เหมาะกับงานแบบเรียลไทม์ เช่น กล้องอัจฉริยะหรือระบบตรวจจับต่าง ๆ ปัจจุบัน YOLOv8 ที่พัฒนาโดยบริษัท Ultralytics เป็นเวอร์ชันที่ถูกใช้งานเยอะ เพราะปรับปรุงทั้งความเร็ว ความแม่นยำ และใช้งานง่ายขึ้น

ในการใช้งานจริงมักใช้ร่วมกับ Ultralytics ซึ่งเป็นไลบรารี Python สำหรับสั่งงาน YOLO ตั้งแต่การเทรนโมเดล ประเมินผล ทดสอบ ไปจนถึงการใช้งานจริง ส่วน OpenCV จะช่วยจัดการภาพและวิดีโอ เช่น รับภาพจากกล้อง ปรับภาพ หรือวาดกรอบแสดงผล ทำให้สามารถทำระบบตรวจจับแบบเรียลไทม์ได้สะดวกขึ้น

YOLOv8 มีหลายขนาดเพื่อให้เหมาะกับงานแต่ละแบบ ได้แก่ YOLOv8n (Nano) ที่เล็กและเร็วมาก เหมาะกับอุปกรณ์สเปกต่ำ, YOLOv8s (Small) เหมาะกับงานทั่วไป, YOLOv8m (Medium) เน้นสมดุลระหว่างความเร็วกับความแม่นยำ, YOLOv8l (Large) ที่เน้นความแม่นยำสูงและมักต้องใช้ GPU และ YOLOv8x (Extra Large) ซึ่งใหญ่ที่สุด เหมาะกับงานระดับสูงที่ต้องการความแม่นยำสูงมาก เช่น ระบบความปลอดภัยหรือการประมวลผลบนเซิร์ฟเวอร์

2.8 เทคนิคเชิงวิศวกรรมศาสตร์

2.8.1 การฝึกโมเดลตรวจจับป้ายทะเบียนด้วย YOLOv8

ชุดข้อมูล (Dataset) ที่ใช้ในการฝึก ชุดข้อมูลที่ใช้ฝึกโมเดลในโครงการนี้เป็นภาพถ่ายป้ายทะเบียนรถยนต์ไทย จัดเก็บไว้ใน Google Drive ภายใต้ชื่อ LicensePlate โดยมีโครงสร้างไดเรกทอรีดังนี้

```
LicensePlate/
  images/
    train/    <- ภาพที่ใช้ฝึก (80%)
    val/      <- ภาพที่ใช้ตรวจสอบ (20%)
  labels/
    train/    <- ไฟล์ Label ในรูปแบบ YOLO (.txt)
    val/
  data.yaml   <- ไฟล์กำหนด Path และชื่อคลาส
```

รูปที่ 2.8 ภาพถ่ายป้ายทะเบียนรถยนต์ไทย จัดเก็บไว้ใน Google Drive

ไฟล์ data.yaml กำหนดคลาสที่ต้องการตรวจจับ ซึ่งในโครงการนี้มีเพียงคลาสเดียว คือ 'Plate' (ป้ายทะเบียน) การแบ่ง Train/Validation Set ทำโดยการสุ่มภาพจาก Training Set 20% แล้วย้ายไปยัง Validation Set โดยใช้สคริปต์ Python ที่เขียนไว้ใน Laptop

ตัวอย่างรูปภาพที่ใช้ในการฝึกโมเดล



รูปที่ 2.9 ตัวอย่างรูปภาพที่ใช้ในการฝึกโมเดล

ภาพที่ใช้ฝึกโมเดลเป็นภาพถ่ายป้ายทะเบียนรถยนต์ไทยที่หลากหลาย ถ่ายในสภาพแสงและมุมกล้องที่แตกต่างกัน พร้อมทั้งมีไฟล์ Label ในรูปแบบ YOLO Format ที่ระบุตำแหน่งและขอบเขตของป้ายทะเบียนในแต่ละภาพ โดยแต่ละไฟล์ Label (.txt)

```
class_id cx cy width height
0 0.512 0.498 0.312 0.089

# class_id = 0 (Plate)
# cx, cy = ตำแหน่งศูนย์กลาง Bounding Box (normalized 0-1)
# width, height = ความกว้างและสูงของ Box (normalized 0-1)
```

รูปที่ 2.10 ไฟล์ Label (.txt)

การใช้ค่า Normalized แทนพิกเซลจริงช่วยให้ Label ไม่ขึ้นอยู่กับขนาดภาพ ทำให้สามารถ Augment (Scale, Resize) ภาพได้โดยไม่ต้องปรับ Label

2.8.2 กระบวนการฝึกโมเดล (Training)

การฝึกโมเดลดำเนินการบน Google Colab โดยใช้ GPU (T4) ผ่าน Ultralytics YOLOv8 library เวอร์ชัน 8.2.103 คำสั่งที่ใช้ฝึกมีดังนี้

```
!yolo task=detect mode=train \
  model=yolov8n.pt \
  data=../data.yaml \
  epochs=250 \
  imgsz=640 \
  augment=True \
  plots=True
```

รูปที่ 2.11 คำสั่งที่ใช้ฝึก

พารามิเตอร์สำคัญ ได้แก่ model=yolov8n.pt ใช้ Pre-trained Weights เป็น Starting Point (Transfer Learning), epochs=250 ฝึก 250 รอบ, imgsz=640 ปรับขนาดภาพเป็น 640×640 pixel ก่อนประมวลผล และ augment=True เปิดใช้ Data Augmentation เพื่อเพิ่มความหลากหลายของข้อมูล

2.8.3 ฟังก์ชัน Loss ที่ใช้ในการฝึก

YOLOv8 ใช้ Loss Function หลัก 3 ตัว ได้แก่

- Box Loss (train/box_loss): วัดความผิดพลาดในการทำนายตำแหน่ง Bounding Box โดยใช้ CloU Loss (Complete Intersection over Union) ซึ่งพิจารณาทั้งขนาด ตำแหน่ง และ อัตราส่วน Aspect Ratio ของ Box
- Class Loss (train/cls_loss): วัดความผิดพลาดในการจำแนกประเภทวัตถุ ใช้ Binary Cross-Entropy Loss สำหรับแต่ละคลาสอิสระ

- DFL Loss (train/df_l_loss): Distribution Focal Loss ช่วยในการทำนายพิกัดขอบของ Bounding Box ให้แม่นยำยิ่งขึ้น โดยกระจาย Probability ไปยัง Discrete Location

ค่า Loss ทั้งหมดนี้ ถูกนำมารวมกันตามน้ำหนักที่กำหนดเพื่อคำนวณ Total Loss สำหรับการ Backpropagation

2.8.4 Metrics ที่ใช้วัดประสิทธิภาพ

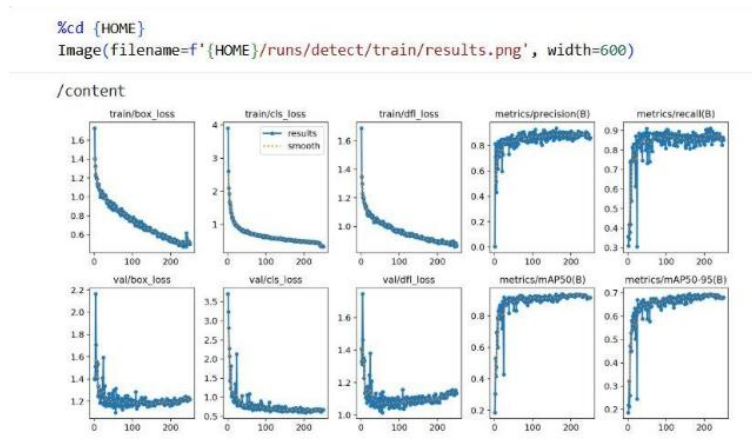
ผลลัพธ์การฝึกโมเดลวัดด้วย Metrics มาตรฐานในงาน Object Detection ดังนี้

- Precision (B): อัตราส่วนของ True Positive ต่อทั้งหมดที่โมเดลทำนายว่าเป็น Positive (TP / (TP + FP)) วัดว่าเมื่อโมเดลบอกว่าเจอป้ายทะเบียน ถูกต้องมากน้อยเพียงใด
- Recall (B): อัตราส่วนของ True Positive ต่อ Positive ทั้งหมดที่มีจริง (TP / (TP + FN)) วัดว่าโมเดลสามารถตรวจจับป้ายทะเบียนที่มีในภาพได้ครบถ้วนมากน้อยเพียงใด
- mAP@50 (B): Mean Average Precision ที่ IoU Threshold = 0.50 คำนี้นับบอกถึงความแม่นยำโดยรวมของโมเดล ค่ายิ่งสูงยิ่งดี
- mAP@50-95 (B): Mean Average Precision เฉลี่ยในช่วง IoU Threshold 0.50–0.95 เป็น Metric ที่เข้มงวดกว่า mAP@50 วัดความสามารถในการ Localize วัตถุอย่างแม่นยำในระดับต่างๆ

2.8.5 ผลการฝึกโมเดล

2.8.5.1 กราฟ Training Metrics

กราฟ Metrics ตลอด 250 Epoch ของการฝึก ครอบคลุมทั้งค่า Loss บน Training Set และ Validation Set รวมถึง Metrics ด้านความแม่นยำ



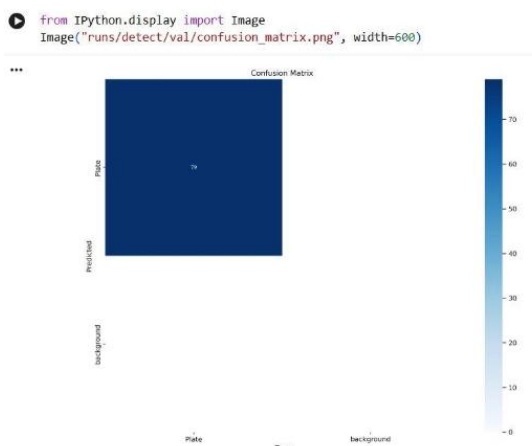
รูปที่ 2.12 กราฟแสดงผล Training Metrics ตลอด 250 Epoch (train/val loss, Precision, Recall, mAP)

จากกราฟในรูปที่ 2.12 สามารถวิเคราะห์ได้ดังนี้

- train/box_loss, train/cls_loss, train/dfl_loss: ค่า Loss ทั้ง 3 ตัวมีแนวโน้มลดลงอย่างต่อเนื่องตลอด 250 Epoch แสดงให้เห็นว่าโมเดลสามารถเรียนรู้จากข้อมูลได้ดี
- val/box_loss, val/cls_loss, val/dfl_loss: ค่า Loss บน Validation Set มีแนวโน้มคงที่และไม่มีการเพิ่มขึ้นอย่างมีนัยสำคัญ แสดงว่าโมเดลไม่เกิด Overfitting
- metrics/precision(B): ค่า Precision บรรจบที่ประมาณ 0.85 หมายความว่าเมื่อโมเดลตรวจจับป้ายทะเบียน มีโอกาส 85% ที่จะถูกต้อง
- metrics/recall(B): ค่า Recall บรรจบที่ประมาณ 0.88 หมายความว่าโมเดลสามารถตรวจจับป้ายทะเบียนที่มีในภาพได้ถึง 88%
- metrics/mAP50(B): ค่า mAP@50 บรรจบที่ประมาณ 0.85 ซึ่งถือว่าดีมากสำหรับงานตรวจจับป้ายทะเบียน
- metrics/mAP50-95(B): ค่า mAP@50-95 บรรจบที่ประมาณ 0.65 แสดงถึงความแม่นยำในการ Localize ที่ดีในระดับ IoU หลายระดับ

2.8.5.2 Confusion Matrix

Confusion Matrix จากการ Validate โมเดลบน Validation Set ซึ่งแสดงความสัมพันธ์ระหว่าง Class ที่โมเดลทำนาย (Predicted) กับ Class ที่แท้จริง (True)



รูปที่ 2.12 Confusion Matrix แสดงผลการ Validate โมเดล YOLOv8 บน Validation Set

จาก Confusion Matrix จะเห็นได้ว่า

- ค่า True Positive (Plate ทำนายว่า Plate) = 79 อินสแตนซ์ แสดงว่าโมเดลตรวจจับป้ายทะเบียนได้ถูกต้องทั้งหมด 79 ภาพ

- ไม่มีการทำนายผิดประเภท (False Positive ต่อคลาส Background) ในเชิงนัยสำคัญ
- Confusion Matrix ที่เป็น Diagonal อย่างชัดเจนบ่งบอกว่าโมเดลมีความสามารถในการแยกแยะป้ายทะเบียนออกจาก Background ได้ดีมาก

2.8.6 การ Export โมเดลและการทำงานบน Edge Device

หลังจากฝึกโมเดลเสร็จสิ้น ไฟล์ Weights ที่ดีที่สุดจะถูกบันทึกไว้ที่ runs/detect/train/weights/best.pt จากนั้นทำการ Export เป็นรูปแบบ ONNX (Open Neural Network Exchange) เพื่อให้สามารถนำไปใช้งานบน Raspberry Pi 4 ได้โดยไม่ต้องพึ่งพา PyTorch

```
from ultralytics import YOLO
model = YOLO('best.pt')
model.export(format='onnx', opset=12, simplify=False)
# ได้ไฟล์ best.onnx สำหรับ Deploy บน RPi4
```

รูปที่ 2.13 การ Export โมเดลและการทำงานบน Edge Device

รูปแบบ ONNX รองรับ Runtime หลากหลาย เช่น ONNX Runtime ที่ใช้ใน Raspberry Pi 4 ช่วยให้ประมวลผลได้เร็วขึ้นด้วย CPU Optimization โดยไม่ต้องติดตั้ง PyTorch ซึ่งมีขนาดใหญ่และอาจทำงานได้ช้ากว่าบน ARM CPU

2.8.7 ระบบอ่านป้ายทะเบียนบน Raspberry Pi 4 (ระบบทางประตูเข้า)

2.8.7.1 ภาพรวมระบบ

ระบบ APAS Entrance ทำงานบน Raspberry Pi 4 เชื่อมต่อกับกล้อง IMX219 (Raspberry Pi Camera Module) ผ่าน CSI Interface โดยมีขั้นตอนการทำงานหลัก 4 ขั้นตอน ได้แก่ (1) ถ่ายภาพด้วยกล้อง (2) ตรวจจับป้ายทะเบียนด้วย YOLOv8 ONNX (3) ประมวลผลภาพและอ่านข้อความด้วย OCR และ (4) ตัดสินใจเปิดไม้กั้นและบันทึกข้อมูลสู่ Firebase

2.8.7.2 การเลือกรูปแบบกล้องและความละเอียดที่เหมาะสม

ก่อนเริ่มการถ่ายภาพ ระบบทำการประเมิน Score ของการตั้งค่ากล้องแต่ละแบบเพื่อเลือกรูปแบบที่ให้คุณภาพภาพดีที่สุดสำหรับการตรวจจับ ผลลัพธ์ที่ได้จากระบบแสดงในตารางที่ 2.2

ตารางที่ 2.2 ค่า Score ของการตั้งค่ากล้อง IMX219 ในรูปแบบต่าง ๆ (ค่าสูง = คุณภาพดีกว่า)

รูปแบบ Sensor	ความละเอียด	Score
SRGGB10_CSI2P	640×480	4093.33
SRGGB10_CSI2P	1640×1232	2557.83
SRGGB10_CSI2P	1920×1080	1250.00
SRGGB10_CSI2P	3280×2464	3275.83
SRGGB8	640×480	5093.33
SRGGB8	1640×1232	3557.83
SRGGB8 (Selected)	1920×1080	2250.00
SRGGB8	3280×2464	4275.83

ระบบเลือกใช้รูปแบบ SRGGB8, 1920×1080 (highlighted ในตาราง) ซึ่งให้ความสมดุลระหว่างความละเอียดและความเร็วในการถ่ายภาพ ทำงานผ่านคำสั่ง rpicas-still โดยบันทึกในรูปแบบ 1280×720 YUV420 สำหรับ Preview

2.8.7.3 Pipeline การตรวจจับและอ่านป้ายทะเบียน

เมื่อได้ภาพจากกล้องแล้ว ระบบจะดำเนินการตามลำดับ 4 ขั้นตอน

1. โมเดล YOLOv8 ที่ Export เป็น ONNX ถูกโหลดผ่าน ONNX Runtime (onnxruntime) บน CPU ภาพจากกล้องจะถูก Preprocess ด้วยการทำ Letterbox (คงอัตราส่วนภาพและ Pad ด้วยสี 114) ให้ได้ขนาด 640×640 pixel จากนั้น Normalize ค่า Pixel เป็น 0–1 ก่อนส่งเข้าโมเดล

```
def letterbox(im, new=640):
    h, w = im.shape[:2]
    r = min(new/h, new/w) # scale ratio
    nh, nw = int(h*r), int(w*r)
    resized = cv2.resize(im, (nw, nh))
    canvas = np.full((new, new, 3), 114, dtype=np.uint8)
    canvas[:nh, :nw] = resized
    return canvas, r # คืน ratio สำหรับแปลง Box กลับ
```

รูปที่ 2.14 define ของ Letterbox

ผลลัพธ์จากโมเดลเป็น Tensor รูปแบบ [N, 5] (cx, cy, w, h, confidence) สำหรับโมเดล Plate ซึ่งมีเพียง 1 คลาส จากนั้นผ่านกระบวนการ Non-Maximum Suppression (NMS) เพื่อเลือก Bounding Box ที่ดีที่สุด โดยใช้ค่า $\text{IOU_THRESHOLD} = 0.45$ และกรองเฉพาะ Detection ที่มี Confidence มากกว่า $\text{CONF_THRESHOLD} = 0.10$

```
def nms(boxes, scores, iou_thres):
    idxs = np.argsort(scores)[::-1] # เรียง Score จากมากไปน้อย
    keep = []
    while len(idxs) > 0:
        i = idxs[0]; keep.append(i)
        # คำนวณ IoU ระหว่าง Box ที่ดีที่สุดกับ Box ที่เหลือ
        xx1 = max(boxes[i][0], boxes[j][0])
        # ... (Intersection over Union)
        # กรองทิ้ง Box ที่ IoU เกิน threshold
```

รูปที่ 2.15 define ของ Non-Maximum Suppression (NMS)

2. Perspective Correction (deskew_plate)

เมื่อตรวจจับป้ายทะเบียนได้แล้ว ระบบจะตัด (Crop) เฉพาะส่วนป้ายทะเบียนออกมา จากนั้นทำการ Deskew เพื่อแก้ไขความเอียงของป้ายทะเบียนที่อาจเกิดจากมุมกล้อง โดยใช้ขั้นตอนดังนี้

- แปลงภาพเป็น Grayscale และทำ Gaussian Blur ลด Noise
- ใช้ Otsu Thresholding แยก Foreground (ตัวอักษร) ออกจาก Background
- หา Contour ที่ใหญ่ที่สุดซึ่งมี 4 มุม (Quadrilateral) แทนขอบป้ายทะเบียน
- ใช้ `cv2.getPerspectiveTransform()` คำนวณ Homography Matrix
- ใช้ `cv2.warpPerspective()` แปลงภาพให้เป็นมุมตรง (Frontal View)

3. Preprocessing สำหรับ OCR

ก่อนส่งภาพป้ายทะเบียนเข้า OCR จะทำการ Preprocessing เพื่อเพิ่มความชัดเจนของตัวอักษร

- ปรับขนาดให้สูงอย่างน้อย 64 pixel (ขยายแบบ Cubic Interpolation)
- CLAHE (Contrast Limited Adaptive Histogram Equalization): ปรับ Contrast เฉพาะพื้นที่ย่อย ทำให้ตัวอักษรสว่างสม่ำเสมอขึ้น
- Bilateral Filter: กำจัด Noise โดยรักษาขอบตัวอักษรไว้
- Otsu Thresholding: แปลงเป็น Binary Image (ขาว-ดำ) เพื่อให้ OCR อ่านได้ง่ายขึ้น

```
def _preprocess_plate(gray):
    if gray.shape[0] < 64:
        gray = cv2.resize(gray, None, fx=64/h, fy=64/h,
                           interpolation=cv2.INTER_CUBIC)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(4,4))
    gray = clahe.apply(gray)
    gray = cv2.bilateralFilter(gray, 9, 75, 75)
    _, binary = cv2.threshold(gray, 0, 255,
                              cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    return binary
```

รูปที่ 2.16 define ของ Preprocessing

4. OCR ด้วย Tesseract

ระบบใช้ Tesseract OCR เป็น OCR Engine หลักสำหรับอ่านข้อความบนป้ายทะเบียน พร้อมทั้งมี PaddleOCR เป็น Primary Engine หากติดตั้งได้สำเร็จ โดยในตัวอย่างผลลัพธ์นี้ใช้ Tesseract

Tesseract ถูกตั้งค่าดังนี้

- --psm 7: Page Segmentation Mode 7 = ถือว่าภาพทั้งหมดเป็น Single Text Line (เหมาะสำหรับป้ายทะเบียน)
- --oem 3: OCR Engine Mode 3 = ใช้ LSTM Neural Network Engine ที่แม่นยำที่สุด
- -c tessedit_char_whitelist: จำกัดให้รู้จักเฉพาะอักขระที่กำหนด คือ พยัญชนะไทย (ก-ฮ) และ ตัวเลข (0-9) เท่านั้น เพื่อลด Error จากการอ่านตัวอักษรอื่น
- lang=thai: ใช้โมเดลภาษาไทยของ Tesseract

2.8.8 กลไกการโหวต (Majority Voting) เพื่อเพิ่มความแม่นยำ

เพื่อลดความผิดพลาดจากการ OCR เพียงครั้งเดียว ระบบทำการอ่านป้ายทะเบียน 3 รอบและใช้ Majority Voting ตัดสินผลสุดท้าย

2.8.8.1 ค่าความมั่นใจของแต่ละอักขระ Tesseract คือน่า Confidence Score ของแต่ละอักขระ (0-100) ซึ่งใช้คำนวณค่าเฉลี่ย (avg_conf) ของผลลัพธ์แต่ละรอบ ตัวอย่างจากการทดสอบกับป้าย 'รถ-2015' แสดงในตารางที่ 2.3 ค่า Confidence เฉลี่ยของรอบนี้ = $(85+80+92+95+91+89) / 6 = 88.67 \approx 88.6\%$

ตารางที่ 2.3 ค่าความมั่นใจของแต่ละอักขระจากการอ่านป้ายทะเบียน รถ2015

อักขระ	ค่าความมั่นใจ	สถานะ
ร	85%	ผ่าน
ถ	80%	ผ่าน
2	92%	ผ่าน
0	95%	ผ่าน
1	91%	ผ่าน
5	89%	ผ่าน

2.8.8.2 ผลการโหวต 3 รอบ

ระบบทำการอ่านและคำนวณ Confidence ใหม่ 3 รอบ แล้วนับคะแนนเสียงของแต่ละผลลัพธ์ แสดงในตารางที่ 2.4

ตารางที่ 2.4 ผลการอ่านป้ายทะเบียนใน 3 รอบและค่าความมั่นใจเฉลี่ยของแต่ละรอบ

รอบที่	ผลลัพธ์ OCR	ค่าความมั่นใจเฉลี่ย
1	รถ2015	88.6%
2	รถ2015	89.1%
3	รด2015	70.2%

ผลการโหวต รถ2015 ได้ 2 คะแนนเสียง, รด2015 ได้ 1 คะแนนเสียง — ผลลัพธ์สุดท้ายคือ รถ2015

```
===== VOTING =====
รถ2015 -> 2 votes
รด2015 -> 1 votes
FINAL RESULT: รถ2015
===== RESULT =====
Predicted   : รถ2015
GroundTruth : รถ2015  ✓
```

รูปที่ 2.17 ผลการโหวต

จะเห็นได้ว่า ในรอบที่ 3 Tesseract อ่านผิดเป็น 'รด2015' (avg_conf=70.2) แต่เนื่องจาก 2 รอบแรกอ่านได้ถูกต้องและมี Confidence สูงกว่า กลไก Majority Voting จึงสามารถแก้ไขข้อผิดพลาดนี้ได้ ทำให้ผลลัพธ์สุดท้ายถูกต้อง

กลไกนี้มีความสำคัญอย่างยิ่งสำหรับการใช้งานจริง เนื่องจาก OCR อาจได้รับผลกระทบจากแสง มุมกล้อง และสภาพป้ายทะเบียน การโหวตหลายรอบช่วยลด False Positive Rate ได้อย่างมีประสิทธิภาพ

2.8.9 การตรวจสอบป้ายทะเบียน (Plate Whitelist)

ระบบตรวจสอบผลลัพธ์ OCR โดยกรองให้เหลือเฉพาะอักขระที่ถูกต้องตาม Whitelist ที่กำหนด ซึ่งประกอบด้วยพยัญชนะไทย ก-ฮ (ยกเว้น ณ ฒ ฎ ฏ ฐ ฑ ฒ ณ ฬ ภ) และตัวเลข 0-9 เท่านั้น

นอกจากนี้ระบบยังใช้ RapidFuzz (fuzzy string matching) เปรียบเทียบผลลัพธ์กับรายการป้ายทะเบียนที่อนุญาต (CarList.txt) และใช้โมเดล YOLOv8 ONNX อีกตัว (CodeProv.onnx) สำหรับตรวจจับรหัสจังหวัดจากส่วนล่างของป้าย

2.8.10 การเชื่อมต่อระบบและการเปิดไม้กั้น

เมื่อระบบตรวจจับและอ่านป้ายทะเบียนสำเร็จ จะดำเนินการดังนี้

- ตรวจสอบจำนวนช่องจอดว่างจาก Firebase Realtime Database (โหนด /tickets)
- สร้าง Ticket ID ในรูปแบบ TK + UUID 8 หลัก เช่น TK4A2F8B1C
- บันทึกข้อมูลป้ายทะเบียน เวลาเข้า และ URL รูปภาพลงใน Firebase
- รอ Raspberry Pi 5 (ระบบจัดการที่จอดรถ) Assign ช่องจอดและสร้าง QR Code
- เมื่อได้รับการยืนยัน ระบบจะสั่งงาน Servo บน GPIO18 เพื่อเปิดไม้กั้น 10 วินาทีแล้วปิดอัตโนมัติ

2.9 จุดเด่นของระบบที่ต่างจากงานทั่วไป

1. ใช้ RPI 2 ตัวทำงานร่วมกันแบบ Double Verification ระบบทั่วไปมักใช้กล้องตัวเดียวอ่านป้ายที่ทางเข้าแล้วจบ แต่ระบบนี้ใช้ RPI 4 อ่านที่ทางเข้า และ RPI 5 อ่านซ้ำในลานจอด แล้วส่งไปให้ Server เปรียบเทียบ ทำให้มีการยืนยันตัวรถยนต์ 2 ชั้น ลดโอกาสรถผิดคันเข้าจอด
2. จัดสรรช่องจอดอัตโนมัติตามลำดับความสำคัญ ระบบทั่วไปแค่บอกว่ามีที่ว่างหรือไม่ แต่ระบบนี้จัดสรรช่องจอดให้เลยโดยอัตโนมัติตามลำดับ A1 ถึง A6 และล็อกช่องนั้นทันทีใน Firebase ไม่ให้ถูกจองซ้ำ
3. ตรวจจับความผิดปกติและแจ้งเตือนด้วย Buzzer หากรถจอดผิดช่องหรือป้ายทะเบียนไม่ตรง Ticket ID ระบบจะแจ้งเตือนทันทีผ่าน Buzzer และพยายามอ่านซ้ำอัตโนมัติสูงสุด 8 ครั้ง ก่อนแสดงสถานะ Error ซึ่งระบบราคาถูกทั่วไปมักไม่มีกลไกนี้
4. อัปเดตสถานะแบบ Real-time ผ่าน Firebase ข้อมูลสถานะช่องจอดทุกช่องถูกซิงค์ขึ้น Cloud ทันที ทำให้สามารถต่อยอดไปสู่แอปพลิเคชันแสดงผลบนมือถือหรือ Web Dashboard ได้ในอนาคต

2.10 งานวิจัยที่เกี่ยวข้อง

วรุฒ พลอยเลี้ยง (2566) ได้พัฒนาระบบตรวจจับตัวอักษรป้ายทะเบียนรถยนต์ด้วยเทคนิคการเรียนรู้เชิงลึก เพื่อเพิ่มประสิทธิภาพและความปลอดภัยในการควบคุมการเข้า-ออกของรถยนต์ในพื้นที่จำกัด โดยใช้ RPI เป็นหน่วยประมวลผลหลัก ร่วมกับการพัฒนาระบบด้วยภาษา Python สำหรับการประมวลผลภาพ และใช้ PyTesseract ในการแปลงภาพป้ายทะเบียนเป็นข้อความ พร้อมบันทึกข้อมูลลงฐานข้อมูลเพื่อจัดเก็บประวัติการเข้า-ออก ผลการทดลองจากรถยนต์จำนวน 20 คัน พบว่าระบบมีความถูกต้องเฉลี่ยประมาณ 90% โดยข้อผิดพลาดเกิดจากปัจจัยด้านแสง ความชัดของป้ายทะเบียน ระยะกล้อง และความคล้ายคลึงของตัวอักษร ซึ่งแสดงให้เห็นว่าระบบสามารถนำไปประยุกต์ใช้ในการลดเวลา เพิ่มความปลอดภัย และพัฒนาต่อยอดในระบบจัดการลานจอดรถอัตโนมัติได้ต่อไป

ภวัต อุดมฐานกุล และ ทรงพล นครเศรษฐ์ (2567) ได้พัฒนาและเพิ่มประสิทธิภาพการตรวจจับป้ายทะเบียนด้วย YOLO และการรู้จำตัวอักษรด้วยแสง (EasyOCR) 2) เพื่อศึกษาและหาเทคนิคและการใช้

อัลกอริทึมที่มีความถูกต้องและมีประสิทธิภาพ เพื่อออกแบบและพัฒนาระบบ ในการวิจัยนี้ใช้เทคนิคการประมวลผลภาพที่ประกอบด้วยการแปลงภาพเป็นระดับสีเทา, การเบลอภาพด้วยฟิลเตอร์เกาส์เซียน, การค้นหาขอบภาพด้วยโซเบล, การวิเคราะห์ภาพไบนารี, การตรวจจับเส้น, และการทำ ความสะอาดภาพด้วยการประมวลผลเชิงมอร์โฟโลยี โดยการทดลองได้ใช้ภาพป้ายทะเบียนทั้งหมดจำนวน 100 ภาพ พบว่าวิธีการที่นำเสนอสามารถตรวจจับและอ่านป้ายทะเบียนได้ความถูกต้องเฉลี่ย 95.26% และ 78.68% ตามลำดับ

กานูวัฒน์ คำจันทร์, ฉันทิศา สกลโอฬาร และปวีรา แพงอุต (2566) ได้ทำการจำลองระบบจอดรถอัตโนมัติ โดยใช้บอร์ด Arduino UNO เป็นตัวควบคุมหลัก ระบบสามารถรองรับรถได้ทั้งหมด 5 คัน และใช้เซ็นเซอร์ Ultrasonic ตรวจวัดระยะเพื่อเช็คว่างช่องจอดนั้นมีรถหรือไม่ จากนั้นบอร์ด Arduino จะประมวลผลข้อมูลที่ได้รับ แล้วสั่งให้ หลอดไฟ LED แสดงสถานะของแต่ละช่องจอดว่า “ว่าง” หรือ “ไม่ว่าง” ทำให้ผู้ใช้งานสามารถมองเห็นสถานะพื้นที่จอดรถได้ง่ายและช่วยจัดการลานจอดให้เป็นระเบียบมากขึ้น

อลงกรณ์ พรสุขศิริ (2565) ได้ศึกษาและประเมิน ประสิทธิภาพของระบบจอดรถอัตโนมัติโดยการเปรียบเทียบระบบจอดรถอัตโนมัติชนิดถาดและ ระบบจอดรถอัตโนมัติชนิดหุ่นยนต์ผ่านการแบบจำลองสถานการณ์ด้วยโปรแกรม SIMIO รวมไปถึง ถึงการวิเคราะห์ต้นทุน และปริมาณการใช้พลังงานไฟฟ้า

กานูวัตร อุทัยบาล และวนิดา แก่นอากาศ (2566) ได้พัฒนาต้นแบบนวัตกรรมของระบบจอดรถอัจฉริยะโดยใช้เทคโนโลยีอินเทอร์เน็ตทุกสรรพสิ่ง (Internet of Things: IoT) พร้อมมาตรฐานเทคโนโลยีวิทยุเครือข่ายบริเวณกว้างที่ใช้พลังงานต่ำ (Narrow Band Internet of Things: NB-IoT) ในการตรวจพบสถานะจุดจอดรถว่ามีสถานะว่างหรือไม่ใช้งาน และส่งไปยังฐานข้อมูลบนเทคโนโลยีคลาวด์ (Cloud Technology) และแสดงสถานะการจอดแบบ เรียลไทม์ผ่านแอปพลิเคชันมือถือ ระบบนี้ช่วยให้ผู้ใช้ได้ทราบตำแหน่งจุดจอดรถที่ว่างแบบเรียลไทม์โดยได้มี การติดตั้งอุปกรณ์ 200 จุดในพื้นที่ศึกษาในพื้นที่จังหวัดขอนแก่น เช่น ในมหาวิทยาลัยขอนแก่น และหน่วยงานราชการในเขตเทศบาลนครขอนแก่น ระบบจอดรถอัจฉริยะนี้ช่วยทำให้เกิดความเข้าใจในพฤติกรรม การใช้พื้นที่จอดรถ ซึ่งข้อมูลนี้เป็นประโยชน์ในการจัดสรรพื้นที่ได้อย่างมีประสิทธิภาพ สร้างรูปแบบการ บริการการจอดรถในวิธีการใหม่ ๆ และการวางแผนการใช้พื้นที่เพื่อสร้างความเจริญเติบโตทางเศรษฐกิจและ ช่วยลดผลกระทบทางสิ่งแวดล้อม

บทที่ 3

วิธีดำเนินงานโครงการ

3.1. ภาพรวมของโครงการ

APMAS คือระบบ ที่ใช้ RPI หลายตัวทำงานร่วมกัน โดยมีส่วนประกอบหลัก 4 ส่วน ดังรูปที่ 3.1

1. Entrance Gate (RPI 4) ประตูทางเข้า

- ใช้ กล้อง RPI Camera Module V2 สำหรับอ่านทะเบียนรถ (LPR - License Plate Recognition)
- RPI4 ส่ง OCR Request ไปยัง Sever เพื่อประมวลผลป้ายทะเบียน
- เมื่อระบบยืนยันแล้ว จะส่งคำสั่ง Open Gate เปิดไม่กั้น
- ข้อมูลรถและสถานะ Ticket ID ถูกส่งไปเก็บที่ Firebase

2. Parking Area 6 ช่องจอด (RPI 5) พื้นที่จอดรถ

- RPI5 ติดตั้งร่วมกับ Ultrasonic Sensor (ตรวจจับรถในช่องจอด) และ กล้อง RPI Camera Module V3
- มี Buzzer และไฟสัญญาณ แจ้งเตือนสถานะช่องจอด
- อัปเดตสถานะช่องจอดกลับไป Firebase แบบ Real-time
- รับข้อมูล LPR Verify และการ Assign ช่องจอด จาก Firebase

3. Exit Gate (ESP 32) ประตูทางออก

- พนักงานตรวจสอบ E-Slip (หลักฐานการชำระเงิน) เทียบกับ Google Sheet
- ถ้าการยืนยันผ่าน → เปิดไม่กั้นให้รถออก

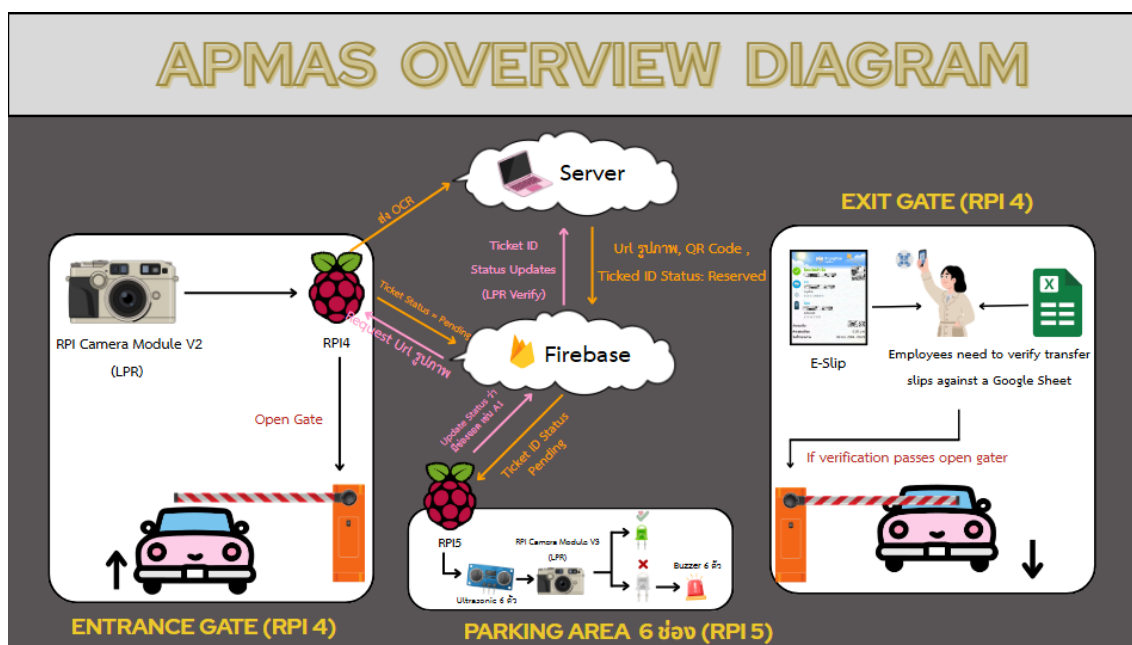
4.ระบบกลาง

- Server รับรู้ป้ายทะเบียน OCR สร้าง Url ภาพและสร้าง QR Code
 - Firebase ทำหน้าที่เป็นฐานข้อมูลแบบเรียลไทม์ (Real-time Database) ซึ่งเป็นศูนย์กลางในการเชื่อมต่อและแลกเปลี่ยนข้อมูลระหว่างอุปกรณ์และระบบต่าง ๆ ภายในโครงการ โดยสามารถแบ่งลักษณะการทำงานออกเป็น 2 ส่วน ดังนี้
1. ข้อมูลที่ถูกส่งเข้าสู่ Firebase

- RPI 4: ส่งข้อมูล Ticket ID และสถานะ (Status) ในรูปแบบ Pending
- Server: ส่งข้อมูล Url ของภาพ และ QR Code
- RPI 5: ส่งข้อมูลผลการตัดสินใจของจอรถ เช่น A1, A2

2. ข้อมูลที่ Firebase ส่งออกไปยังระบบต่าง ๆ

- Server: รับข้อมูล Ticket ID ที่มีการอัปเดตสถานะใหม่
- RPI 4: รับข้อมูลสถานะ (Status) ที่ถูกอัปเดตล่าสุด
- RPI 5: รับข้อมูลหมายเลขป้ายทะเบียน (License Plate Recognition: LPR) ที่ผ่านการตรวจสอบ (Verify) แล้ว



รูปที่ 3.1 APMAS Overview Diagram

3.2. โครงสร้างระบบซอฟต์แวร์โดยรวม

APMAS ประกอบด้วยซอฟต์แวร์ที่ทำงานบนอุปกรณ์หลายตัวพร้อมกัน โดยใช้ Firebase เป็นศูนย์กลางในการแลกเปลี่ยนข้อมูลระหว่างอุปกรณ์ทุกชิ้น อุปกรณ์แต่ละตัวไม่จำเป็นต้องสื่อสารตรงกัน แต่จะเขียนและอ่านข้อมูลผ่าน Firebase แทน ยกเว้นกรณีที่ต้องส่งคำสั่งให้ Server ประมวลผล ซึ่งจะใช้ HTTP API โดยตรง องค์ประกอบของระบบซอฟต์แวร์ทั้งหมดมีดังตารางที่ 3.1

ตารางที่ 3.1 องค์ประกอบของระบบซอฟต์แวร์ทั้งหมด

อุปกรณ์	ซอฟต์แวร์/ไฟล์	หน้าที่หลัก
RPI 5	Parking_lot_v2.py (Python)	ควบคุมลานจอดรถ Sensor 6 ช่องจอด, OCR ป้ายทะเบียน, จัดช่องจอด, LED, Buzzer, GUI
RPI 4	Main-GUI.py (Python)	ควบคุมประตูทางเข้า: OCR ป้ายทะเบียน, Firebase, Servo Motor ไม้กั้น, แสดง Slip + QR Code
Server (FastAPI)	รันที่ 10.65.185.199:8000	สร้าง QR Code, ตรวจสอบป้ายทะเบียน, เก็บรูปภาพที่อัปโหลดจาก RPI 4
Firebase Realtime DB	asia-southeast1	ตัวกลาง Realtime ระหว่างทุกอุปกรณ์ เก็บ Tickets และ Slots
Google Sheet	Google	บันทึกข้อมูลการชำระเงิน พนักงานทางออกตรวจสอบก่อนเปิดไม้กั้น
Admin UI	admin.html (Web)	เจ้าหน้าที่ดูสถานะ Tickets ช่อง, แก้ไขป้ายทะเบียน, พิมพ์ Slip แบบ Realtime
GitHub Pages	index.html (Web)	ผู้ใช้สแกน QR Code ดูข้อมูลตัวและชำระเงินผ่าน PromptPay

3.3. โครงสร้างข้อมูลใน Firebase

ข้อมูลใน Firebase แบ่งเป็น 2 โฟลเดอร์หลัก

3.3.1 โฟลเดอร์ Tickets เก็บข้อมูลรถแต่ละคันที่เข้าใช้บริการ โดยใช้ Ticket ID เป็น Key เช่น TK18494F75 ดังตารางที่ 3.2

ตารางที่ 3.2 โฟลเดอร์ tickets เก็บข้อมูลรถแต่ละคันที่เข้าใช้บริการ

Field	ตัวอย่างค่า	คำอธิบาย
ticket_id	TK18494F75	รหัสตัวที่สร้างโดย RPI4
plate_text_raw	4กล7774	ป้ายทะเบียนที่ OCR อ่านได้
image_plate_url	http://10.65.185.199:8000/images/...	Url รูปป้ายทะเบียน
qr_url	http://10.65.185.199:8000/images/TK18494F75.png	Url รูป QR Code

ตารางที่ 3.2 (ต่อ) โพลเดอร์ tickets เก็บข้อมูลรถแต่ละคันที่เข้าใช้บริการ

Field	ตัวอย่างค่า	คำอธิบาย
slot_id	A4	ช่องจอดที่ได้รับ
source	entrance_pi4	แหล่งที่มาของข้อมูล
status	exited	สถานะปัจจุบัน
time_in	2026-03-17T19:52:39.898205	เวลาที่รถเข้า
time_out	2026-03-17T20:44:10.794141	เวลาที่รถออก
ticket_id	TK18494F75	รหัสตัวที่สร้างโดย RPI 4

สถานะ (status) ของ ticket เปลี่ยนแปลงตามขั้นตอนการทำงาน ดังตารางที่ 3.3

ตารางที่ 3.3 สถานะ (status) ของ ticket เปลี่ยนแปลงตาม

status	ความหมาย	อัปเดตโดย
pending	RPI 4 เพิ่งสร้าง Ticket รอ RPI5 จัดช่อง	RPI 4
slot_assigned	RPI 5 จัดช่องจอดให้แล้ว รอ Server สร้าง QR	RPI 5
verified	Server สร้าง QR เสร็จแล้ว พร้อมให้รถเข้า	Server
parked	RPI 5 ยืนยันป้ายตรงแล้ว รถจอดในช่องจอดแล้ว	Server
exited	รถออกจากช่องแล้ว ระบบ Reset เสร็จ	RPI 5

3.3.2. โพลเดอร์ slots เก็บสถานะช่องจอดปัจจุบัน A1 ถึง A6

ตารางที่ 3.4 โพลเดอร์ slots เก็บสถานะช่องจอด

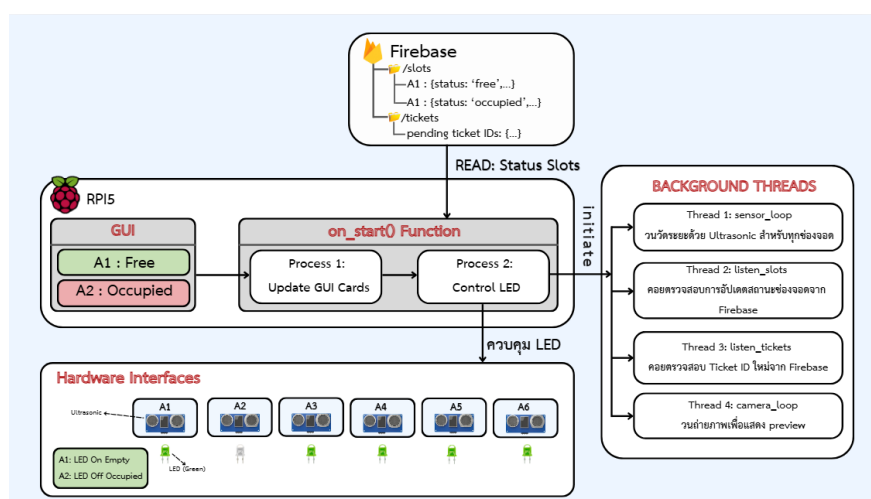
Field	ตัวอย่างค่า	คำอธิบาย
status	Free / Reserved / Occupied	สถานะช่อง
ticket_id	TK18494F75	Ticket ที่ใช้ช่องจอดนี้อยู่
plate	4กล7774	ป้ายทะเบียนที่จอดอยู่
time_in	2026-03-17T19:52:39	เวลาที่รถเข้าจอด

3.4. ลำดับการทำงานของระบบ

ขั้นตอนที่ 1: เริ่มระบบ (RPI 5 ส่วนลานจอด)

เมื่อทำการเปิดโปรแกรม Parking_lot_v2.py ระบบจะเรียกใช้ฟังก์ชัน on_start() ซึ่งเป็นฟังก์ชันหลักสำหรับการเริ่มต้นการทำงานทั้งหมดของระบบ ดังแสดงในรูปที่ 3.2 โดยมีรายละเอียดการทำงานดังนี้

- ทำการอ่านข้อมูลสถานะของช่องจอดรถจาก Firebase ในส่วน Slots เพื่อนำมาแสดงผลบน GUI ในรูปแบบการ์ดแสดงสถานะของแต่ละช่องจอด
- ควบคุมการทำงานของหลอดไฟ LED ตามสถานะของช่องจอด โดยกำหนดให้ช่องที่มีสถานะ Occupied (มีรถจอด) ไฟ LED จะดับ และช่องที่ว่างไฟ LED จะติด
- เริ่มต้นการทำงานแบบขนาน (Multithreading) ในรูปแบบ Background Thread จำนวน 4 กระบวนการ ได้แก่
 - sensor_loop: ทำหน้าที่ตรวจวัดระยะของแต่ละช่องจอดอย่างต่อเนื่อง
 - listen_slots: ทำหน้าที่รับการเปลี่ยนแปลงสถานะของช่องจอดจาก Firebase แบบเรียลไทม์
 - listen_tickets: ทำหน้าที่รับข้อมูล Ticket ใหม่จาก Firebase
 - camera_loop: ทำหน้าที่ถ่ายภาพอย่างต่อเนื่องเพื่อแสดงผลแบบ Preview



รูปที่ 3.2 การทำงานของ RPI 5 ขณะเริ่มระบบ

ข้อมูลที่แสดงบนหน้าจอ ณ ขั้นตอนนี้ (อ่านจาก Firebase)

/slots/A1: { status: "free", ticket_id: null, plate: null }

/slots/A2: { status: "free", ticket_id: null, plate: null }

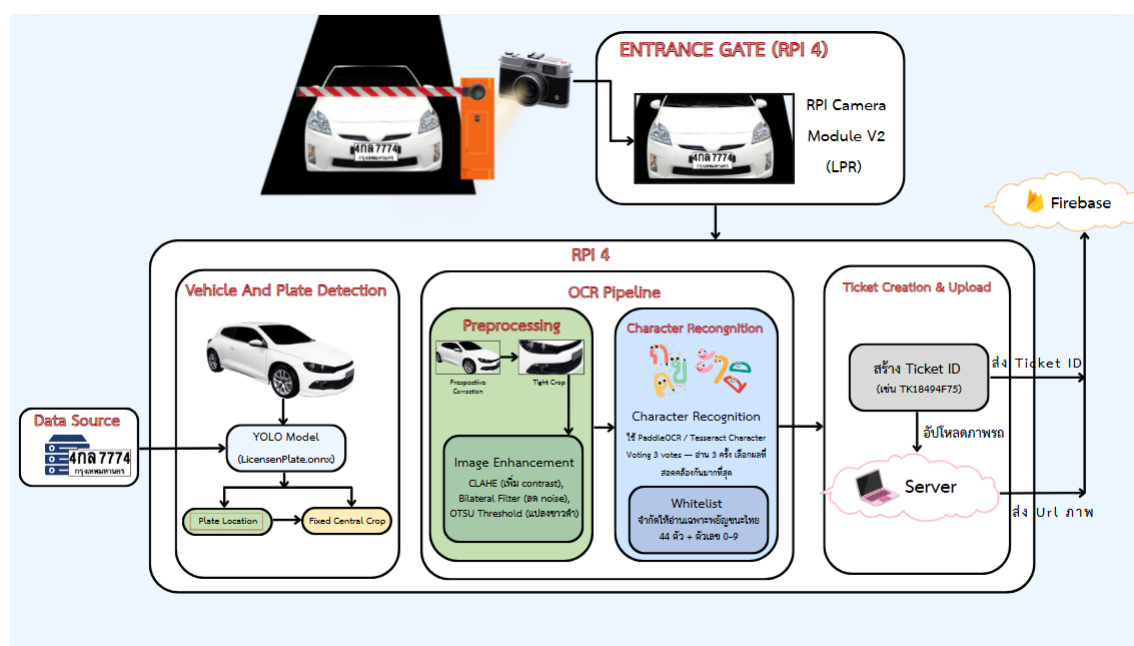
/slots/A3: { status: "occupied", ticket_id: "TKxxxx", plate: "กข1234" }

ขั้นตอนที่ 2: ตรวจจบบรรทัดและอ่านป้ายทะเบียน

เมื่อรถขับเข้ามาที่ประตูทางเข้า กล้อง RPI Camera Module V2 (LPR) บน RPI4 จะถ่ายรูป รูปที่ถ่ายได้จะถูกส่งเข้า OCR Pipeline ซึ่งประกอบด้วยขั้นตอนดังรูปที่ 3.3

- โมเดล YOLO (LicensePlate.onnx) สแกนรูปทั้งใบเพื่อหาตำแหน่งป้ายทะเบียน และ Crop เฉพาะส่วนป้ายออกมา ถ้าหาไม่พบจะใช้ fixed crop บริเวณกลางรูปแทน
- ทำ Perspective Correction แก้ความเอียงของป้าย และ Tight Crop ตัดขอบขาวรอบป้ายออก
- ปรับปรุงคุณภาพภาพด้วย CLAHE เพิ่ม contrast, Bilateral Filter ลด noise และ OTSU Threshold แปลงเป็นภาพขาวดำ
- ส่งภาพเข้า Tesseract อ่านตัวอักษร โดย vote ผล 3 ครั้งแล้วเลือกผลที่ตรงกันมากที่สุด ใช้ whitelist เฉพาะพยัญชนะไทย 44 ตัวและตัวเลข 0-9

เมื่ออ่านป้ายได้แล้ว RPI4 สร้าง Ticket ID แบบสุ่มไปให้ Firebase เช่น TK18494F75 และอัปโหลดรูปป้ายทะเบียนไปเก็บที่ Server



รูปที่ 3.3 การทำงานของระบบตรวจจบบรรทัดและอ่านป้ายทะเบียน

ข้อมูลที่อัปเดตใน Firebase ณ ขั้นตอนนี้

```
/tickets/TK18494F75: {
  ticket_id:      "TK18494F75"
  plate_text_raw: "4กส7774"
  image_plate_url: "http://10.65.185.199:8000/images/TK18494F75_plate.jpg"
  source:         "entrance_pi4"
  time_in:        "2026-03-17T19:52:39.898205"
  slot_id:        "pending"
  status:         "pending"
  qr_url:         ""
}
```

ขั้นตอนที่ 3: จัดช่องจอดและอัปเดตสถานะ

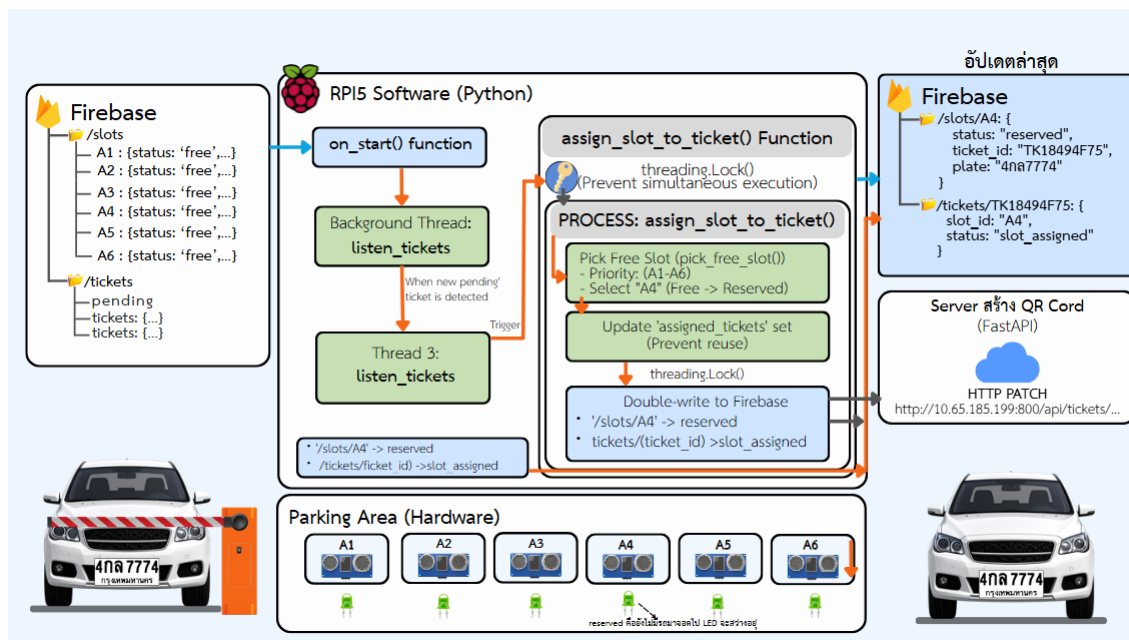
RPI 5 ทำหน้าที่รับข้อมูลจาก Firebase ใน Object ที่มีชื่อว่า Tickets แบบเรียลไทม์ ผ่านกลไก Listener ซึ่งเป็นตัวรับฟังเหตุการณ์ที่ทำงานอย่างต่อเนื่อง เมื่อระบบตรวจพบ Ticket ID ใหม่ ที่มีสถานะเป็น Pending และยังไม่ได้รับการกำหนด slot_id ระบบจะสร้างกระบวนการทำงานแบบขนาน (Thread) เพื่อเรียกใช้ฟังก์ชัน assign_slot_to_ticket() สำหรับจัดสรรช่องจอดให้กับ Ticket ดังแสดงในรูปที่ 3.4

เพื่อป้องกันการทำงานซ้ำซ้อน ระบบมีการใช้กลไก Lock (Threading.Lock) ในการควบคุมการเข้าถึงข้อมูลร่วม (Shared Resource) เพื่อไม่ให้เกิดการจัดสรรช่องจอดซ้ำสำหรับ Ticket เดียวกัน

ขั้นตอนการจัดสรรช่องจอด

- ตรวจสอบข้อมูลใน assigned_tickets ซึ่งเป็นชุดข้อมูล (Set) ของ Ticket ที่ได้รับการจัดสรรช่องจอดแล้ว โดยใช้ Threading.Lock() เพื่อป้องกันการแก้ไขข้อมูลพร้อมกันจากหลาย Thread
- เรียกใช้ฟังก์ชัน pick_free_slot() เพื่อเลือกช่องจอดที่ว่าง โดยทำการอ่านข้อมูลจาก Key Slots ใน Firebase และเลือกช่องที่มีสถานะเป็น Free ตามลำดับ (เช่น A1 ถึง A6)
- ทำการอัปเดตข้อมูลไปยัง Firebase พร้อมกัน 2 ส่วน ได้แก่
 - ใน Object Slots: ปรับสถานะเป็น reserved (ถูกจองแล้ว)
 - ใน Object Tickets: ปรับสถานะเป็น slot_assigned (ได้รับการจัดสรรช่องจอดแล้ว)

- ส่งคำสั่ง HTTP PATCH ไปยัง Server เพื่อร้องขอให้ระบบทำการสร้าง QR Code สำหรับ Ticket นั้น



รูปที่ 3.4 การทำงานของระบบจัดช่องจอดและอัปเดตสถานะ

ข้อมูลที่อัปเดตใน Firebase ณ ขั้นตอนนี้

```
/slots/A4: {
  status: "reserved"
  ticket_id: "TK18494F75"
  plate: "4กถ7774"
}

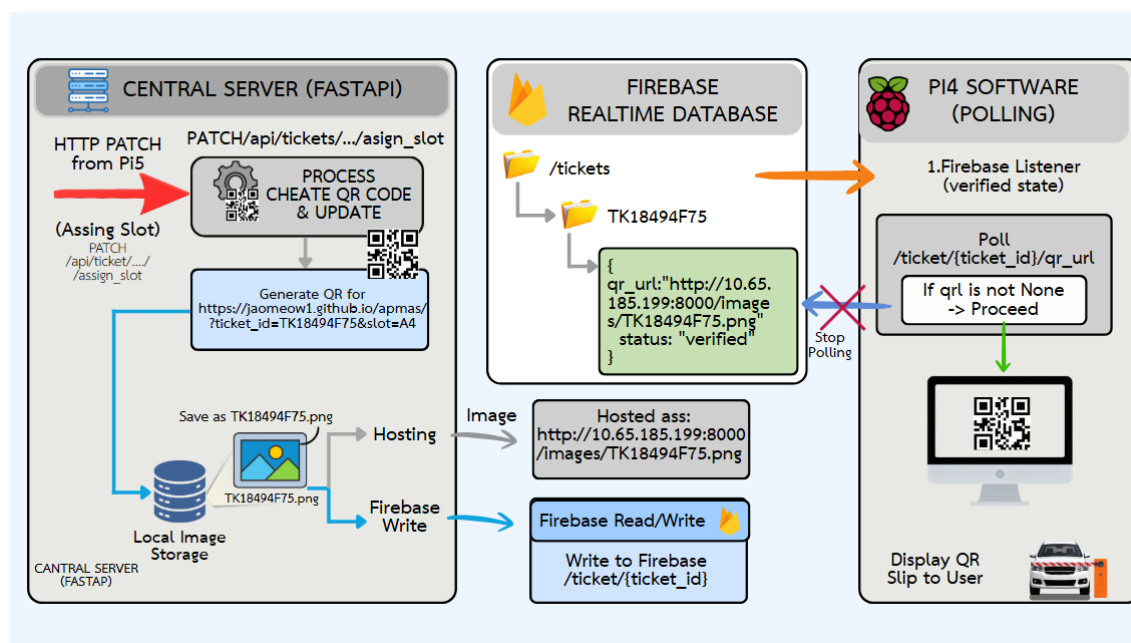
/tickets/TK18494F75: {
  slot_id: "A4"
  status: "slot_assigned"
}
```

จากนั้น RPi5 แจ้ง Server ผ่าน HTTP PATCH:

PATCH `http://10.65.185.199:8000/api/tickets/TK18494F75/assign_slot?slot_id=A4`

ขั้นตอนที่ 4 Server สร้าง QR Code

Server รับคำสั่งจาก RPI5 แล้วสร้าง QR Code ที่ประกอบด้วย Url สำหรับผู้ใช้สแกนชำระเงิน: https://jaomeow1.github.io/apmas/?ticket_id=TK18494F75&slot=A4 ดังรูปที่ 3.5



รูปที่ 3.5 การทำงานของระบบ Server สร้าง QR Code

Server บันทึกไฟล์รูป QR และอัปเดต Firebase:

/tickets/TK18494F75: {

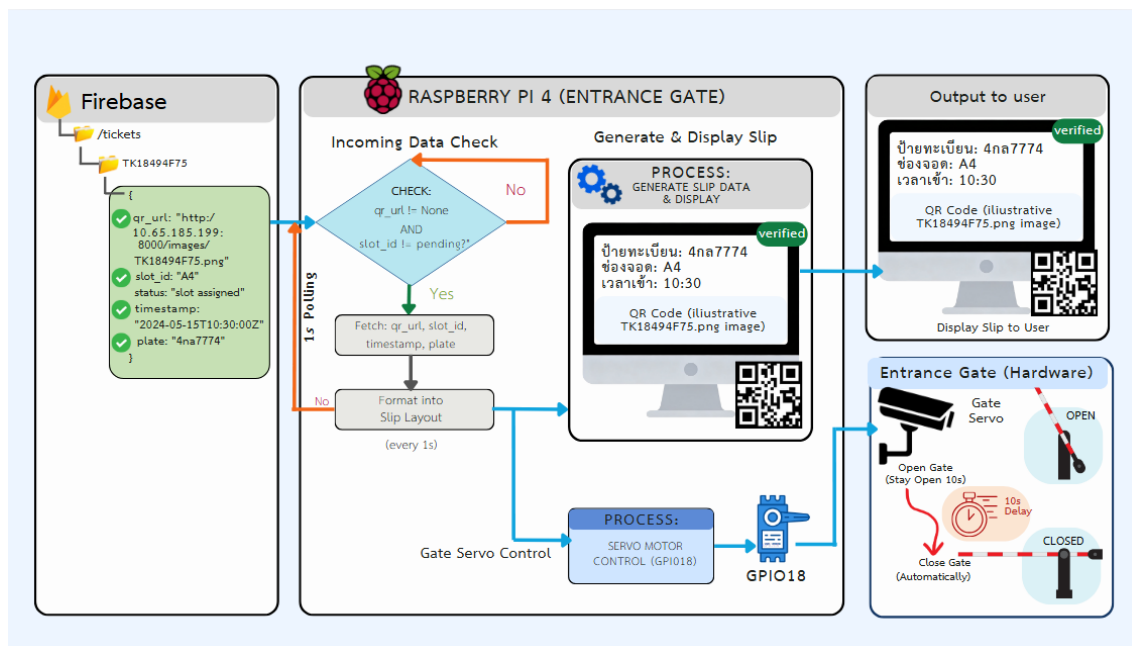
qr_url: "http://10.65.185.199:8000/images/TK18494F75.png"

status: "verified"

}

ขั้นตอนที่ 5 เปิดไม้กั้นและแสดง Slip

RPI4 Poll Firebase (คอยเช็คข้อมูลจาก Firebase ซ้ำ ๆ เป็นช่วงเวลา) จะเช็คทุก 1 วินาที รอจนกว่า Key qr_url จะมีค่าและ Key slot_id ไม่ใช่ pending เมื่อพร้อมแล้ว RPI4 แสดง Slip บนหน้าจอส่วนทางเข้าซึ่งประกอบด้วยป้ายทะเบียน ช่องจอดที่ได้รับ เวลาเข้า และ QR Code สำหรับชำระเงิน พร้อมกันนั้นสั่ง Servo Motor บน GPIO18 เปิดไม้กั้นค้างไว้ 10 วินาทีแล้วปิดอัตโนมัติ ดังรูปที่ 3.6



รูปที่ 3.6 การทำงานของระบบเปิดไม้กั้นและแสดง Slip

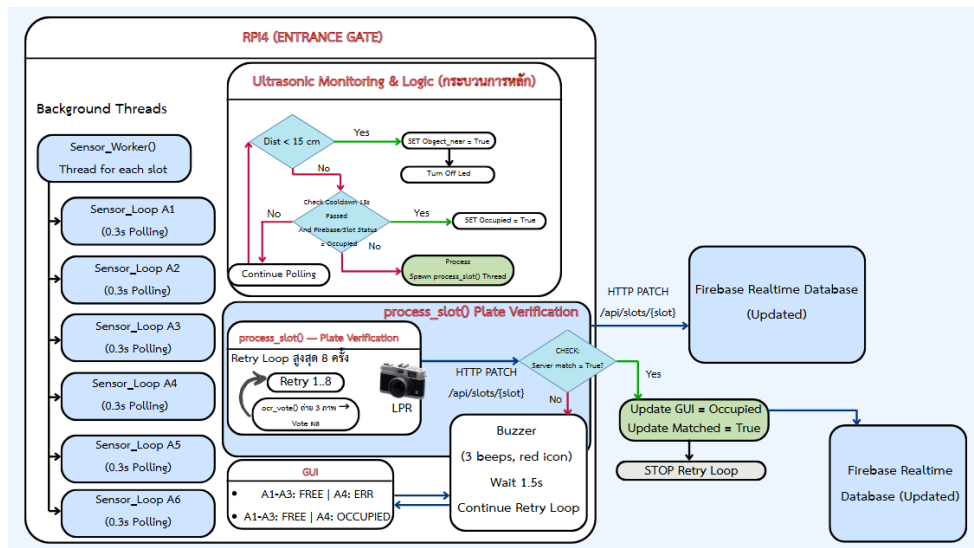
ขั้นตอนที่ 6 ตรวจสอบการจอดในช่อง

Ultrasonic แต่ละช่องทำงานใน thread (งานย่อยแยกขนาน) แยกกัน ผ่าน `_sensor_worker()` (ฟังก์ชันทำงานหลักของ sensor แต่ละช่อง) โดย Poll (ตรวจสอบซ้ำๆ) ระยะทุก 0.3 วินาที มี Logic (เงื่อนไขการตัดสินใจ) ดังรูปที่ 3.7

กรณีวัตถุเข้าใกล้ ($\text{dist} < \text{DETECT_CM} = 15 \text{ cm}$)

- ดับ LED ทั้งที่และตั้ง `object_near` (ตัวแปรบอกว่ามีวัตถุอยู่ใกล้ช่อง) = True
- เมื่อรถเข้าใกล้ Ultrasonic จะทำการนับถอยหลัง 15 วินาที เมื่อผ่านเงื่อนไขการตรวจสอบป้ายทะเบียนระบบจะอัปเดตเป็น Occupied ไปที่ Firebase Slots Status
- Retry (ลองซ้ำ) ถ่ายรูปและ OCR (การอ่านตัวอักษรจากภาพ) สูงสุด 8 ครั้ง โดย Vote (เลือกเสียงข้างมาก) 3 รูปต่อครั้งด้วย `ocr_vote()` (ฟังก์ชันถ่ายหลายรูปแล้วเลือกผลที่ดีที่สุด)
- ส่งผลป้ายทะเบียนให้ Server ตรวจสอบผ่าน HTTP PATCH `/api/slots/{slot}`
- Server ค้นหา Ticket ที่ `slot_id` (รหัสช่องจอด) ตรงกัน แล้วเปรียบเทียบป้ายทะเบียน
- หาก Match = True (ป้ายตรงกัน): อัปเดต GUI (หน้าจอแสดงผล) = occupied, บันทึก `matched=True` (ยืนยันสำเร็จ), หยุด Retry

- หาก Match = False (ป้ายไม่ตรง): Buzzer ดัง 3 สั้น, รอ 1.5 วินาที, Retry ต่อ
- เมื่อตรวจสอบครบ 8 ครั้งโดยไม่ Match จะแสดง GUI สถานะ ERR (ผิดพลาด)



รูปที่ 3.7 การทำงานของระบบการตรวจสอบการจอดในช่อง

ข้อมูลที่อัปเดตใน Firebase เมื่อ Match สำเร็จ:

`/slots/A4: { status: "occupied", time_in: "2026-03-17T19:52:39" }`

`/tickets/TK18494F75: { status: "parked" }`

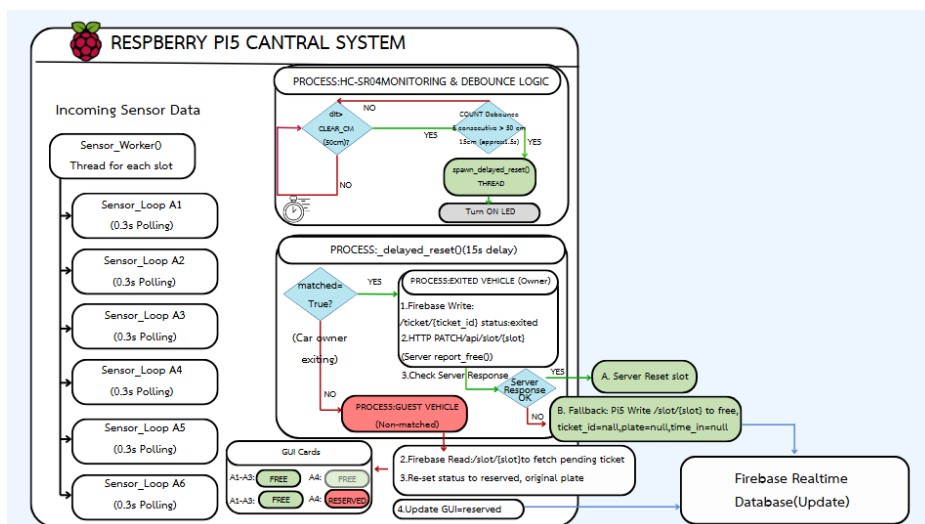
ขั้นตอนที่ 7 รถออกจากช่องจอด และระบบ Reset

เมื่อ Ultrasonic ตรวจพบว่าระยะมากกว่า `CLEAR_CM = 25 cm` (ระยะขั้นต่ำที่ถือว่าช่องว่าง) ระบบจะนับ Debounce (การกรองสัญญาณผิดพลาด) โดยต้องอ่านค่า `> 25 cm` ติดต่อกัน 5 ครั้ง (~1.5 วินาที) จึงถือว่ารถออกจริง จากนั้น Spawn Thread (สร้างงานย่อยใหม่) `_delayed_reset()` (ฟังก์ชัน Reset ที่หน่วงเวลาไว้ก่อน) รอ 15 วินาที ดังรูปที่ 3.8

กรณี Matched = True (รถเจ้าของออกยืนยันป้ายสำเร็จแล้ว)

- อัปเดต Firebase `/Tickets` ไปยัง `status: exited` (ออกแล้ว), `time_out` จะมีค่าเป็นเวลาปัจจุบัน
- เรียก `report_free(slot)` (ฟังก์ชันรายงานช่องจอดว่างแล้ว) ส่งไปที่ `HTTP PATCH /api/slots/{slot}` เพื่อให้ Server Reset ช่อง

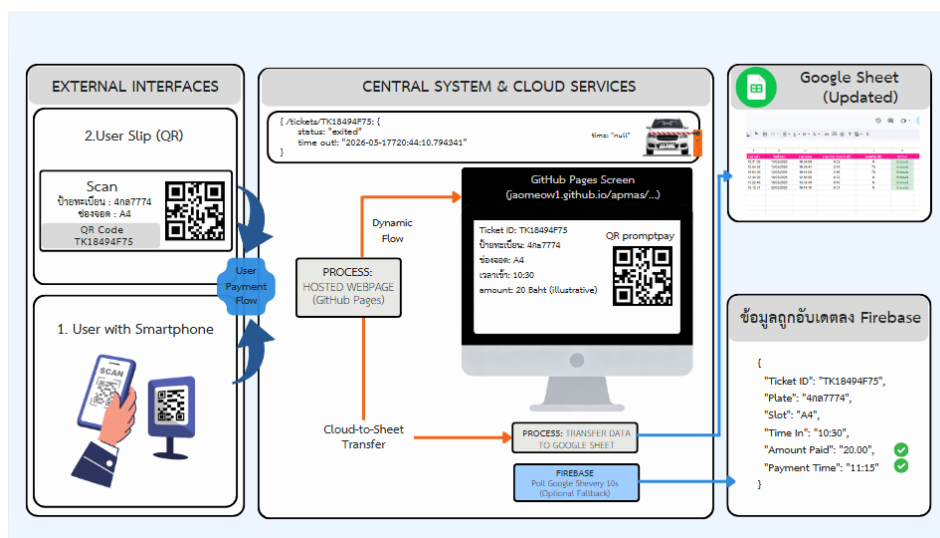
- หาก Server ไม่ตอบสนอง RPI5 จะเขียน Firebase โดยตรงเป็น fallback (ทางเลือกสำรอง)
กรณี Matched = False (รถอื่นเข้ามาแล้วออกไป — ยังไม่เคยยืนยันป้ายสำเร็จ)
- Reset ค่าใน plate_checked (สถานะตรวจป้ายแล้ว) และ ค่าใน Matched (สถานะจับคู่สำเร็จ)
- อ่านข้อมูล Object Slots มี Key Slots จาก Firebase เพื่อดึง Ticket ID เดิม ที่ยังมีค่า reserved (จองอยู่) อยู่
- คืนสถานะ GUI กลับเป็น reserved (จอง) พร้อมป้ายทะเบียนเดิม



รูปที่ 3.8 การทำงานของระบบรถออกจากช่องจอด และระบบ Reset

ขั้นตอนที่ 8 ผู้ใช้ชำระเงินผ่าน QR Code

ระหว่างที่รถจอดอยู่ในลานจอด ผู้ใช้สแกน QR Code จาก Slip ที่แสดงบนหน้าจอทางเข้า จะพาไปที่หน้าเว็บ GitHub Pages ซึ่งแสดงข้อมูล Ticket ID ป้ายทะเบียน ช่องจอด เวลาเข้า และช่องทางชำระเงินผ่าน PromptPay เมื่อผู้ใช้ชำระเงินเสร็จสิ้น ข้อมูลจะถูกบันทึกลง Google Sheet อัตโนมัติ ประกอบด้วย Ticket ID, ป้ายทะเบียน, ช่องจอด, เวลาเข้า, จำนวนเงินที่ชำระ และเวลาที่ชำระดังรูปที่ 3.9



รูปที่ 3.9 การทำงานของระบบผู้ใช้ชำระเงินผ่าน QR Code

ขั้นตอนที่ 8 การทำงานของประตูทางออก (Exit Gate)

เมื่อผู้ใช้ต้องการนำรถออกจากลานจอดรถ จะต้องผ่านกระบวนการดังตารางที่ 3.5

ตารางที่ 3.5 กระบวนการนำรถออกจากลานจอดรถ

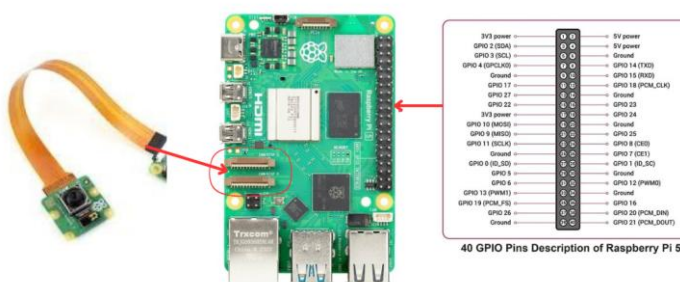
ขั้นที่	ผู้ดำเนินการ	รายละเอียด
1	ผู้ใช้	ชำระค่าจอดรถผ่าน QR PromptPay บนหน้าเว็บก่อนนำรถออก ระบบบันทึกสถานะลง Google Sheet อัตโนมัติ
2	ผู้ใช้	แสดง Slip ที่มี QR Code ให้แก่พนักงานที่ประตูทางออก (QR มี ticket ID ฝังอยู่)
3	พนักงาน	สแกน QR Code จาก Slip ระบบนำ Ticket ID ไปตรวจสอบกับ Google Sheet
4	ระบบ	เปรียบเทียบ: Ticket ID ตรงกัน, ป้ายทะเบียนตรงกัน, สถานะการชำระเงินเป็น 'ชำระแล้ว'
5	พนักงาน	หากข้อมูลถูกต้องครบถ้วน สั่งให้ระบบเปิด Servo Motor ยกไม้กั้นทางออก

3.3.2 ขั้นตอนของฮาร์ดแวร์

ส่วนลานจอตรง

รูปที่ 3.10 แสดงการเชื่อมต่อระหว่าง กล้อง RPI Camera Module V3 กับบอร์ด RPI 5 ซึ่งเป็นอุปกรณ์หลักในส่วนลานจอตรงของระบบ โดยกล้องจะถูกเชื่อมต่อเข้ากับบอร์ดผ่านสายแพ (Ribbon Cable) ไปยังพอร์ตเฉพาะบนบอร์ดที่ออกแบบมาสำหรับกล้องโดยตรง บริเวณที่วงกลมสีแดงในภาพ แสดงตำแหน่งของพอร์ตเชื่อมต่อกล้องบนบอร์ด RPI 5 ซึ่งทำหน้าที่รับข้อมูลภาพจากกล้องเพื่อนำไปประมวลผล เช่น การแสดงภาพแบบเรียลไทม์ (Preview) หรือการใช้งานร่วมกับระบบตรวจจับป้ายทะเบียนรถ นอกจากนี้ ทางด้านขวาของภาพยังแสดง ผังขา (GPIO Pin Diagram) ของ RPI 5 จำนวน 40 ขา ซึ่งใช้สำหรับเชื่อมต่อกับอุปกรณ์ภายนอกอื่น ๆ เช่น เซ็นเซอร์ Ultrasonic, หลอดไฟ LED และอุปกรณ์ควบคุมต่าง ๆ ภายในระบบลานจอตรง

โดยสรุป ภาพนี้แสดงให้เห็นทั้งการเชื่อมต่อกล้องสำหรับรับข้อมูลภาพ และโครงสร้างขา GPIO ที่ใช้ในการควบคุมอุปกรณ์ต่าง ๆ ซึ่งเป็นส่วนสำคัญในการทำงานของระบบทั้งหมด



รูปที่ 3.10 ต่อกล้องเข้ากับบอร์ด RPI 5 และ Pin ทั้งหมด

(ที่มา: <https://hackatronic.com/raspberry-pi-5-pinout-specifications-pricing-a-complete-guide/> (บอร์ดและพิน) และ <https://www.waveshare.com/raspberry-pi-camera-module-3> (กล้อง))

การต่อของ Ultrasonic Sensor 6 ตัว

ในการติดตั้ง Ultrasonic Sensor จำนวน 6 ตัว เพื่อใช้ตรวจจับระยะของแต่ละช่องจอด จำเป็นต้องคำนึงถึงระดับแรงดันไฟฟ้าของสัญญาณที่ส่งกลับมายังบอร์ด RPI 5 โดยขา Echo ของเซ็นเซอร์จะส่งสัญญาณแรงดันที่ระดับ 5 โวลต์ ในขณะที่ขา GPIO ของ RPI 5 รองรับแรงดันได้สูงสุดเพียง 3.3 โวลต์เท่านั้น

ดังนั้น เพื่อป้องกันความเสียหายที่อาจเกิดขึ้นกับบอร์ด จึงต้องมีการใช้งาน วงจรแบ่งแรงดันไฟฟ้า (Voltage Divider) ที่ขา Echo ของเซ็นเซอร์ทุกตัว โดยกำหนดให้ใช้ตัวต้านทานค่า $R1 = 1$ กิโลโอห์ม ต่อแบบอนุกรม และ $R2 = 2$ กิโลโอห์ม ต่อไปยังกราวด์ (GND) เพื่อปรับระดับแรงดันจาก

5V ให้ลดลงอยู่ในช่วงที่ RPI 5 สามารถรับได้อย่างปลอดภัย รายละเอียดการเชื่อมต่อของ Ultrasonic Sensor ทั้ง 6 ตัว แสดงดังตารางที่ 3.6

ตารางที่ 3.6 การต่อ Ultrasonic Sensor 6 ตัว

ช่องจอด	Trig (GPIO Output)	Echo (GPIO Input)
A1	GPIO 23	GPIO 24
A2	GPIO 16	GPIO 25
A3	GPIO 5	GPIO 19
A4	GPIO 12	GPIO 13
A5	GPIO 6	GPIO 26
A6	GPIO 20	GPIO 21

การต่อของ LED สีเขียว 6 ตัว

ในการแสดงสถานะของช่องจอดรถ ระบบได้ติดตั้ง LED สีเขียว จำนวน 6 ตัว โดยกำหนดให้ทำงานในรูปแบบ Active HIGH กล่าวคือ เมื่อมีการส่งค่าสัญญาณลอจิก 1 (HIGH) ไปยังขา GPIO จะทำให้ LED ติดสว่าง และเมื่อส่งค่า 0 (LOW) จะทำให้ LED ดับ

ในขั้นตอนการเริ่มต้นระบบ จะมีการสั่งงานให้ LED ทุกดวงติดสว่างทันที โดยใช้คำสั่ง `gpio_write(h, pin, 1)` ซึ่งหมายความว่า เมื่อเปิดระบบครั้งแรก ทุกช่องจอดจะถูกกำหนดให้มีสถานะเป็น “ว่าง” และแสดงผลด้วยไฟสีเขียวที่ติดอยู่ทุกช่อง

วิธีการเชื่อมต่อวงจร LED

การเชื่อมต่อ LED แต่ละตัวจะใช้ตัวต้านทานเพื่อจำกัดกระแสไฟฟ้า โดยมีรูปแบบการต่อดังนี้

- ต่อขา GPIO ผ่านตัวต้านทานค่า 220 โอห์ม ไปยังขา Anode (+) ของ LED และต่อขา Cathode (-) ลงสู่กราวด์ (GND) เพื่อให้วงจรสามารถทำงานได้อย่างถูกต้องและปลอดภัย โดยรูปแบบการเชื่อมต่อนี้จะถูกนำไปใช้เหมือนกันกับ LED ทั้งหมดจำนวน 6 ตัว เพื่อให้การทำงานของวงจรมีความสม่ำเสมอและปลอดภัยต่ออุปกรณ์

รายละเอียดการเชื่อมต่อของ LED สีเขียวทั้ง 6 ตัว แสดงดังตารางที่ 3.7

ตารางที่ 3.7 การต่อ LED สีเขียว 6 ตัว

ช่องจอด	GPIO (Output)
A1	GPIO 2
A2	GPIO 3
A3	GPIO 4
A4	GPIO 14
A5	GPIO 15
A6	GPIO 18

การต่อของ Buzzer 6 ตัว

ในการแจ้งเตือนข้อผิดพลาดของระบบ เช่น กรณีตรวจพบว่าหมายเลขป้ายทะเบียนไม่ตรงกับข้อมูลที่กำหนด ระบบได้ติดตั้ง Buzzer จำนวน 6 ตัว โดยควบคุมการทำงานผ่านสัญญาณแบบ PWM (Pulse Width Modulation) จากขา GPIO ของ RPI 5 การสร้างสัญญาณเสียงใช้คำสั่ง `lgpio.tx_pwm` โดยกำหนดความถี่ที่ 1200 Hz และ Duty Cycle ที่ 50% เพื่อให้เกิดเสียงที่ชัดเจน ทั้งนี้ Buzzer จะทำงานเมื่อเกิดเหตุการณ์ผิดพลาด (ฟังก์ชัน `beep_err`) โดยมีรูปแบบการส่งสัญญาณเสียงดังนี้

- ส่งสัญญาณเสียงที่ความถี่ 1200 Hz จำนวน 3 ครั้ง
- แต่ละครั้งมีระยะเวลา 0.25 วินาที
- เว้นช่วงหยุดระหว่างแต่ละครั้ง 0.10 วินาที

วิธีการเชื่อมต่อวงจร Buzzer

การเชื่อมต่อ Buzzer แต่ละตัวสามารถทำได้โดยตรงจากขา GPIO โดยมีรูปแบบดังนี้

- เชื่อมต่อขา GPIO เข้ากับขา Buzzer (+) และเชื่อมต่อขา Buzzer (-) ลงสู่กราวด์ (GND) เพื่อให้วงจรสามารถทำงานได้อย่างถูกต้อง

โดยรูปแบบการเชื่อมต่อนี้จะใช้เหมือนกันกับ Buzzer ทั้งหมดจำนวน 6 ตัว เพื่อให้ระบบสามารถควบคุมการแจ้งเตือนได้อย่างมีประสิทธิภาพ

หมายเหตุ: หากใช้ Buzzer ที่ต้องการกระแสสูง อาจจำเป็นต้องเพิ่มทรานซิสเตอร์เพื่อช่วยขับสัญญาณ รายละเอียดการเชื่อมต่อของ Buzzer ทั้ง 6 ตัว แสดงดังตารางที่ 3.8

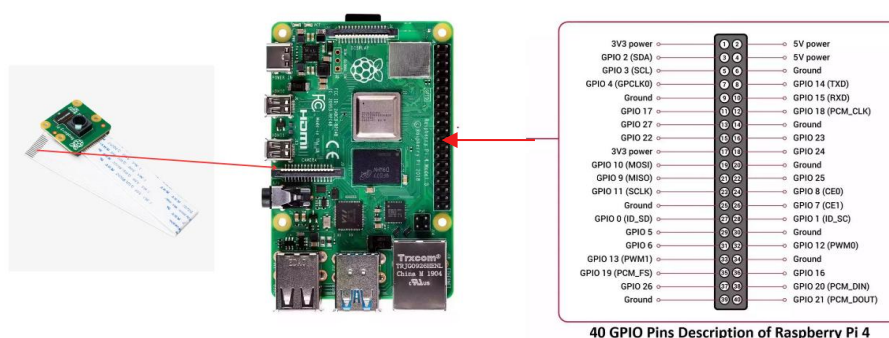
ตารางที่ 3.8 การของ Buzzer 6 ตัว

ช่องจอด	GPIO (PWM Output)
A1	GPIO 27
A2	GPIO 17
A3	GPIO 22
A4	GPIO 10
A5	GPIO 9
A6	GPIO 11

ส่วนทางเข้า

รูปที่ 3.11 แสดงการเชื่อมต่อระหว่าง RPI Camera Module V2 กับบอร์ด RPI 4 โดยใช้สายแพ (Ribbon Cable) เชื่อมต่อเข้ากับพอร์ตสำหรับกล้องบนบอร์ด ซึ่งเป็นพอร์ตเฉพาะที่ใช้สำหรับรับข้อมูลภาพโดยตรง

ตำแหน่งที่มีลูกศรชี้ในภาพ แสดงจุดเชื่อมต่อของสายกล้องเข้ากับบอร์ด RPI 4 ซึ่งทำหน้าที่รับข้อมูลภาพจากกล้องเพื่อนำไปประมวลผล เช่น การบันทึกภาพ หรือการใช้งานร่วมกับระบบตรวจจับป้ายทะเบียนรถ ทางด้านขวาของภาพเป็นผังขา GPIO จำนวน 40 ขา ของ RPI 4 ซึ่งใช้สำหรับเชื่อมต่อกับอุปกรณ์ภายนอกต่าง ๆ เช่น เซ็นเซอร์ หลอดไฟ LED และอุปกรณ์ควบคุมอื่น ๆ ภายในระบบ โดยสรุป ภาพนี้แสดงให้เห็นโครงสร้างการเชื่อมต่อกล้องเข้ากับบอร์ด RPI 4 และผังขา GPIO ที่ใช้สำหรับควบคุมอุปกรณ์ต่าง ๆ ซึ่งเป็นส่วนสำคัญในการทำงานของระบบทั้งหมด



รูปที่ 3.11 การต่อกล้องเข้ากับบอร์ด RPI 4 และ Pin ทั้งหมด

(ที่มา: <https://www.hackatronic.com/raspberry-pi-4-specifications-pin-diagram-and-description/> (บอร์ดและพิน) และ <https://quartzcomponents.com/products/raspberry-pi-camera-module-2> (กล้อง))

การต่อของ Servo Motor (ไม้กั้นทางเข้า) RPI 4

ในการควบคุมไม้กั้นทางเข้า ระบบใช้ Servo Motor ซึ่งสามารถควบคุมมุมการหมุนได้อย่างแม่นยำ โดยเชื่อมต่อเข้ากับบอร์ด RPI 4 ผ่านขา GPIO เพื่อส่งสัญญาณควบคุมแบบ PWM (Pulse Width Modulation)

การเชื่อมต่อ Servo Motor ประกอบด้วย 3 สายหลัก ดังนี้

- สายสัญญาณ (Signal) สีส้ม/เหลือง เชื่อมต่อกับขา GPIO 18 ของ RPI 4 ซึ่งเป็นขาที่รองรับสัญญาณ PWM เพื่อใช้ควบคุมมุมการหมุนของ Servo Motor
- สายไฟเลี้ยง (VCC) สีแดง เชื่อมต่อกับขา 5V ของบอร์ด เพื่อจ่ายพลังงานให้กับ Servo Motor
- สายกราวด์ (GND) สีน้ำตาลหรือสีดำ เชื่อมต่อกับขา GND ของบอร์ด เพื่อเป็นจุดอ้างอิงแรงดันไฟฟ้า

ในการทำงาน ระบบจะส่งสัญญาณ PWM จากขา GPIO 18 เพื่อกำหนดตำแหน่งของ Servo Motor เช่น การยกหรือปิดไม้กั้นทางเข้า โดยสามารถควบคุมผ่านตัวแปรในโปรแกรม

รายละเอียดการเชื่อมต่อ Servo Motor แสดงดังตารางที่ 3.9

ตารางที่ 3.9 การต่อ Servo Motor (ไม้กั้นทางเข้า) RPI 4

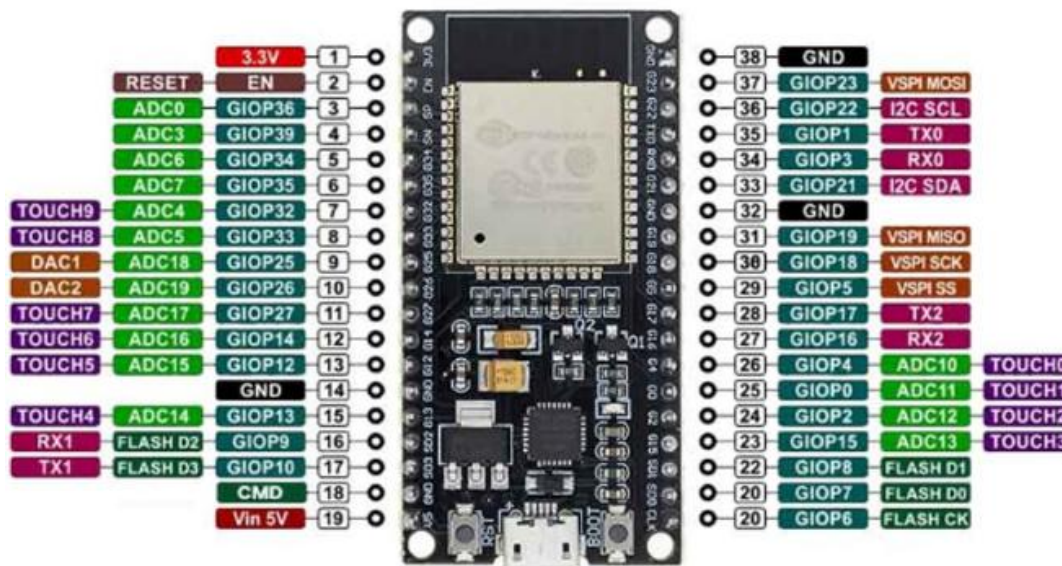
อุปกรณ์	ขา RPI 4
Servo Motor	GPIO 18

ส่วนทางออก

รูปที่ 3.14 แสดงผังขา (Pin Diagram) ของบอร์ด ESP32 ซึ่งเป็นไมโครคอนโทรลเลอร์ที่ใช้ในการควบคุมอุปกรณ์ต่าง ๆ ภายในระบบ โดยในภาพจะแสดงตำแหน่งและหน้าที่ของขาแต่ละขาอย่างชัดเจน ขาของ ESP32 สามารถแบ่งตามหน้าที่หลักได้ดังนี้

- ขาไฟเลี้ยง (Power): เช่น 3.3V และ 5V ใช้สำหรับจ่ายพลังงานให้กับบอร์ดและอุปกรณ์ภายนอก
- ขากราวด์ (GND): ใช้เป็นจุดอ้างอิงแรงดันไฟฟ้า
- ขา GPIO (General Purpose Input/Output): ใช้สำหรับรับ-ส่งสัญญาณดิจิทัลเพื่อควบคุมอุปกรณ์ เช่น LED, Relay หรือ Sensor
- ขาสัญญาณพิเศษ: เช่น
 - ADC (Analog to Digital Converter): ใช้รับค่าสัญญาณอนาล็อกจากเซ็นเซอร์
 - PWM: ใช้ควบคุมอุปกรณ์ที่ต้องการปรับระดับ เช่น มอเตอร์หรือความสว่างของ LED
 - I2C, SPI, UART: ใช้สำหรับการสื่อสารกับอุปกรณ์อื่น

โดยภาพนี้ช่วยให้สามารถเลือกใช้งานให้เหมาะสมกับอุปกรณ์ต่าง ๆ ในระบบได้อย่างถูกต้อง เช่น การเลือกขาที่รองรับ PWM สำหรับควบคุมมอเตอร์ หรือขา ADC สำหรับอ่านค่าจากเซ็นเซอร์



รูปที่ 3.12 Pin ทั้งหมด ของ ESP 32

(ที่มา: <https://www.ortech-online.com/product/1886/cb001-บอร์ด-esp32-38pin-type-c-esp32s-chip-cp2102-wifi-bluetooth-board-esp-wroom-32-nodemcu>)

การต่อของ Servo Motor (ไม้กั้นทางเข้า) ESP 32

ในการควบคุมไม้กั้นทางออก ระบบใช้ Servo Motor ซึ่งสามารถควบคุมมุมการหมุนได้อย่างแม่นยำ โดยเชื่อมต่อเข้ากับบอร์ด ESP32 ผ่านขา GPIO เพื่อส่งสัญญาณควบคุมแบบ PWM (Pulse Width Modulation)

การเชื่อมต่อ Servo Motor ประกอบด้วย 3 สายหลัก ดังนี้

- สายสัญญาณ (Signal) สีส้ม/เหลือง เชื่อมต่อกับขา GPIO ของ ESP32 (ขาที่รองรับ PWM) เพื่อใช้ควบคุมมุมการหมุนของ Servo Motor
- สายไฟเลี้ยง (VCC) สีแดง เชื่อมต่อกับขา 5V หรือ Vin ของบอร์ด เพื่อจ่ายพลังงานให้กับ Servo Motor
- สายกราวด์ (GND) สีน้ำเงินหรือสีดำ เชื่อมต่อกับขา GND ของบอร์ด เพื่อเป็นจุดอ้างอิงแรงดันไฟฟ้า

ในการทำงาน ระบบจะส่งสัญญาณ PWM จากขา GPIO ของ ESP32 เพื่อกำหนดตำแหน่งของ Servo Motor เช่น การยกหรือปิดไม้กั้นทางออก โดยสามารถควบคุมผ่านตัวแปรในโปรแกรมได้ตามที่กำหนด

รายละเอียดการเชื่อมต่อ Servo Motor แสดงดังตารางที่ 3.10

ตารางที่ 3.10 การต่อ Servo Motor (ไม้กั้นทางเข้า) ESP 32

อุปกรณ์	ขา Pi4
Servo Motor	GPIO 19

บทที่ 4

ผลการทดสอบ

4.1 การทดสอบความแม่นยำในการตรวจจับป้ายทะเบียน (ทางเข้า)

การทดสอบนี้มีวัตถุประสงค์เพื่อประเมินประสิทธิภาพของระบบอ่านป้ายทะเบียนที่ติดตั้งบน RPI 4 โดยใช้กล้อง RPI Camera Module V2 ร่วมกับโมเดล YOLO สำหรับตรวจจับตำแหน่งป้ายทะเบียน และ Tesseract สำหรับแปลงภาพเป็นตัวอักษรและตัวเลขของป้ายทะเบียนรถยนต์ในประเทศไทย

วิธีการทดสอบ

ในการทดสอบ ได้ใช้ป้ายทะเบียนจำนวน 6 ป้าย โดยทำการทดสอบป้ายละ 3 ครั้ง รวมทั้งสิ้น 18 ครั้ง ภายใต้สภาพแสงปกติในเวลากลางวัน และกำหนดระยะห่างระหว่างกล้องกับป้ายทะเบียนประมาณ 10 เซนติเมตร

ผลการทดสอบจะถูกบันทึกในรูปแบบของความถูกต้องในการอ่านป้ายทะเบียน โดยพิจารณาว่าระบบสามารถอ่านตัวอักษรและตัวเลขได้ตรงกับป้ายทะเบียนจริงหรือไม่ ซึ่งแสดงรายละเอียดดัง ตารางที่ 4.1

ตารางที่ 4.1 ผลการทดสอบอ่านป้ายทะเบียนทางเข้า

		ผลที่ระบบอ่านได้		
ครั้งที่	ป้ายทะเบียนจริง	รอบที่ 1	รอบที่ 2	รอบที่ 3
1	7กจ 3572	7กจ 3572 ✓	7กจ 3572 ✓	7กจ 3572 ✓
2	กก 2563	กก 2563 ✓	กก 25630 ✗	กก 2563 ✓
3	วร 8765	วร 8765 ✓	วร 8765 ✓	วร 8765 ✓
4	กท 2058	กท 2058 ✓	กท 2058 ✓	กท 2058 ✓
5	ชย 6555	ชย 6555 ✓	ชย 6555 ✓	ชย 6555 ✓
6	รธ 2015	รธ 2015 ✓	รธ 2015 ✓	รธ 2015 ✓

ผลการทดสอบ

พบว่าระบบสามารถอ่านป้ายทะเบียนได้อย่างถูกต้องในรอบที่ 1 ครบทั้ง 6 ครั้ง คิดเป็นความแม่นยำร้อยละ 100 สำหรับรอบที่ 2 ระบบเกิดข้อผิดพลาดในการทดสอบครั้งที่ 2 โดยอ่านป้ายทะเบียน "กก 2563" ผิดเป็น "กก 25630" เมื่อวิเคราะห์สาเหตุของความผิดพลาดที่เกิดขึ้น พบว่ามีปัจจัยที่ส่งผลต่อความแม่นยำในการอ่านป้ายทะเบียนคือ สภาพแสงและสภาพแวดล้อม หากแสงในบริเวณทางเข้ามีความเข้มไม่สม่ำเสมอ เช่น แสงสะท้อนจากพื้นผิวป้ายทะเบียน หรือแสงแดดโดยตรงที่ส่องมายังกล้อง อาจทำให้เกิดความแตกต่าง (contrast) ของภาพที่ไม่เหมาะสม ส่งผลให้ระบบ

ตรวจจับขอบเขตของตัวอักษรและตัวเลขได้ไม่ชัดเจน และอาจตีความเงาหรือ artifact ที่ปรากฏในภาพว่าเป็นส่วนหนึ่งของตัวเลข ซึ่งในกรณีนี้ทำให้ระบบอ่านเลข "3" ต่อท้ายจนกลายเป็น "25630" แทนที่จะเป็น "2563" ความแม่นยำในรอบที่ 2 อยู่ที่ร้อยละ 83.33 และในรอบที่ 3 ระบบสามารถอ่านป้ายทะเบียนได้ถูกต้องครบทั้ง 6 ครั้ง คิดเป็นความแม่นยำร้อยละ 100

เมื่อพิจารณาภาพรวมทั้ง 3 รอบ จากการทดสอบทั้งหมด 18 ครั้ง (6 ครั้ง × 3 รอบ) ระบบอ่านถูกต้อง 17 ครั้ง มีข้อผิดพลาดเพียง 1 ครั้ง คิดเป็นความแม่นยำโดยรวมร้อยละ 94.44

4.2 การทดสอบความแม่นยำในการจับคู่ป้ายทะเบียนระหว่างทางเข้ากับลานจอดรถ

การทดสอบนี้ตรวจสอบว่าป้ายทะเบียนที่ RPi4 อ่านได้ตอนเช้าสามารถจับคู่ (match) กับป้ายที่ RPi5 อ่านได้ในลานจอดรถได้ถูกต้องหรือไม่ เนื่องจากทั้งสองตัวทำงานแยกกันและอาจอ่านได้ผลต่างกันเล็กน้อยจากสภาพแสงและมุมกล้องที่ต่างกัน

วิธีการทดสอบ

การทดสอบระบบเปรียบเทียบป้ายทะเบียนดำเนินการโดยนำป้ายทะเบียนจำนวน 6 ป้าย ขับเข้าสู่ระบบตามขั้นตอนปกติ โดยเริ่มจาก RPi 4 ทำหน้าที่อ่านป้ายทะเบียนที่บริเวณทางเข้า จากนั้น RPi 5 จะอ่านป้ายทะเบียนอีกครั้งเมื่อรถยนต์เคลื่อนเข้าสู่ลานจอด และส่งข้อมูลทั้งสองชุดไปยัง Server เพื่อทำการเปรียบเทียบผล โดยระบบจะบันทึกว่าสามารถจับคู่ข้อมูลได้สำเร็จหรือไม่ในแต่ละรอบ ทั้งนี้กำหนดเงื่อนไขว่าหากระบบไม่สามารถจับคู่ได้สำเร็จจะดำเนินการอ่านซ้ำ โดยจำกัดจำนวนรอบสูงสุดไม่เกิน 8 รอบต่อป้ายทะเบียน ซึ่งแสดงรายละเอียดดัง ตารางที่ 4.2

ตารางที่ 4.2 ผลการจับคู่ป้ายทะเบียนระหว่างทางเข้ากับลานจอดรถ

คันที่	ป้ายจาก RPi4 (ทางเข้า)	ป้ายจาก RPi5 (ลานจอดรถ)	จำนวนครั้งที่ลอง จับคู่
1	7กจ 3572	7กจ 3572	2
2	กก 2563	กก 2563	1
3	วร 8765	วร 8765	2
4	กท 2058	กท 2058	2
5	ชย 6555	ชย 6555	4
6	รท 2015	รท 2015	1

ผลการทดสอบ

ผลการทดสอบพบว่าระบบสามารถจับคู่ป้ายทะเบียนได้สำเร็จครบทั้ง 6 ป้าย คิดเป็นอัตราความสำเร็จร้อยละ 100 โดยป้ายทะเบียน กก 2563 และป้ายทะเบียน รล 2015 ใช้จำนวนรอบในการจับคู่ที่น้อยที่สุด คือ 1 รอบ ในขณะที่ป้ายทะเบียน 7กจ 3572 ป้ายทะเบียน วร 8765 และป้ายทะเบียน กท 2058 ใช้จำนวน 2 รอบ และป้ายทะเบียน ขย 6555 ใช้จำนวนรอบมากที่สุดคือ 4 รอบ ซึ่งยังอยู่ภายในเกณฑ์ที่กำหนดไว้ไม่เกิน 8 รอบ โดยเฉลี่ยแล้วระบบใช้จำนวน 2 รอบในการจับคู่ป้ายทะเบียนต่อคัน

จากผลการทดลองข้างต้นแสดงให้เห็นว่าระบบเปรียบเทียบป้ายทะเบียนมีความน่าเชื่อถือและสามารถทำงานได้อย่างถูกต้องภายใต้เงื่อนไขการทดสอบที่กำหนด อย่างไรก็ตาม การที่ป้ายทะเบียนบางคันต้องใช้หลายรอบในการจับคู่อาจเกิดจากปัจจัยด้านคุณภาพของภาพหรือสภาพแสงในลานจอดที่แตกต่างจากบริเวณทางเข้า ซึ่งส่งผลให้ระบบต้องดำเนินการอ่านซ้ำก่อนที่จะจับคู่ข้อมูลได้อย่างแม่นยำ

4.3 การทดสอบการตัดสินใจเลือกช่องจอดรถ

การทดสอบในส่วนนี้มีวัตถุประสงค์เพื่อตรวจสอบความสามารถของ RPI 5 ในการเลือกช่องจอดรถที่ว่างได้อย่างถูกต้องตามลำดับความสำคัญที่กำหนดไว้ ตั้งแต่ช่อง A1 ไปจนถึง A6 ตามลำดับได้ถูกต้องหรือไม่

วิธีการทดสอบ

กรณีที่ 1 ทดสอบในสถานะที่ช่องจอดทุกช่องว่างพร้อมกัน ตั้งแต่ช่อง A1 ถึง A6 โดยส่งรถยนต์เข้าที่ละคันและบันทึกว่าระบบเลือกช่องจอดตามลำดับที่กำหนดหรือไม่ และเมื่อช่องจอดใดถูกจองแล้วระบบจะไม่นำช่องนั้นมาจัดสรรให้รถคันถัดไปอีก

กรณีที่ 2 ทดสอบในสถานะที่มีเฉพาะช่องจอดหมายเลขคู่ว่าง ได้แก่ ช่อง A2 A4 และ A6 โดยช่องหมายเลขคี่ A1 A3 และ A5 ถูกจองอยู่ก่อนแล้ว เพื่อตรวจสอบว่าระบบสามารถข้ามช่องที่ถูกจองและเลือกช่องจอดว่างที่มีลำดับความสำคัญสูงสุดในขณะนั้นได้ถูกต้องหรือไม่

กรณีที่ 3 ทดสอบในสถานะที่มีเฉพาะช่องจอดหมายเลขคี่ว่าง ได้แก่ ช่อง A1 A3 และ A5 โดยช่องหมายเลขคู่ A2 A4 และ A6 ถูกจองอยู่ก่อนแล้ว เพื่อตรวจสอบว่าระบบเลือกช่องจอดตามลำดับความสำคัญได้ถูกต้องในสถานะที่ช่องว่างไม่ต่อเนื่องกัน

ในทุกกรณี บันทึกผลว่าระบบเลือกช่องจอดได้ถูกต้องตามลำดับที่กำหนดหรือไม่ ซึ่งแสดงรายละเอียดดัง ตารางที่ 4.3

ตารางที่ 4.3 ผลการทดสอบการเลือกช่องจอด

กรณีที่ 1 (ป้ายทะเบียน)	ช่องที่ว่าง	ช่องที่ระบบเลือก (RPI 5)
วร 8765	A1 – A6	A1
กก 2563	A2 – A6	A2
รธ 2015	A3 – A5	A3
7กจ 3572	A4 – A6	A4
กท 2058	A5 – A6	A5
ชย 6555	A6	A6
กรณีที่ 2 (ป้ายทะเบียน)	ช่องที่ว่าง	ช่องที่ระบบเลือก (RPI 5)
7กจ 3572	A2, A4, A6	A2
กท 2058	A4, A6	A4
รธ 2015	A6	A6
กรณีที่ 3 (ป้ายทะเบียน)	ช่องที่ว่าง	ช่องที่ระบบเลือก (RPI 5)
กท 2058	A1, A3, A5	A1
วร 8765	A3, A5	A3
7กจ 3572	A5	A5

ผลการทดสอบ

กรณีที่ 1 ทดสอบในสถานะที่ช่องจอดทุกช่องว่างพร้อมกันตั้งแต่ช่อง A1 ถึง A6 โดยส่งรถยนต์เข้าที่ละคันจำนวน 6 คัน พบว่าระบบเลือกช่องจอดได้ถูกต้องตามลำดับความสำคัญในทุกครั้ง กล่าวคือ รถคันแรกได้รับการจัดสรรช่อง A1 รถคันที่สองได้ช่อง A2 และลำดับถัดไปจนถึงรถคันที่หกซึ่งได้รับช่อง A6 ตามลำดับ แสดงให้เห็นว่าระบบสามารถเลือกช่องจอดที่มีลำดับความสำคัญสูงสุดได้อย่างถูกต้อง และเมื่อช่องใดถูกจองแล้วระบบจะไม่นำช่องนั้นมาจัดสรรซ้ำอีก

กรณีที่ 2 ทดสอบในสถานะที่มีเฉพาะช่องจอดหมายเลขคู่ว่าง ได้แก่ ช่อง A2 A4 และ A6 โดยช่องหมายเลขคี่ถูกจองไว้ก่อนแล้ว พบว่าระบบสามารถข้ามช่องที่ถูกจองและเลือกช่องว่างที่มีลำดับความสำคัญสูงสุดในขณะนั้นได้ถูกต้องทุกครั้ง โดยรถคันแรกได้รับช่อง A2 รถคันที่สองได้ช่อง A4 และรถคันที่สามได้ช่อง A6 ตามลำดับ

กรณีที่ 3 ทดสอบในสถานะที่มีเฉพาะช่องจอดหมายเลขคี่ว่าง ได้แก่ ช่อง A1 A3 และ A5 โดยช่องหมายเลขคู่ถูกจองไว้ก่อนแล้ว พบว่าระบบสามารถเลือกช่องจอดได้ถูกต้องเช่นเดียวกัน โดย

รถคันแรกได้รับช่อง A1 รถคันที่สองได้ช่อง A3 และรถคันที่สามได้ช่อง A5 ตามลำดับ แสดงให้เห็นว่า แม้ช่องว่างจะไม่ต่อเนื่องกัน ระบบก็ยังคงสามารถจัดลำดับและเลือกช่องที่ถูกต้องได้

โดยภาพรวมทั้ง 3 กรณี ระบบสามารถเลือกช่องจอดได้ถูกต้องครบทุกการทดสอบ คิดเป็นความแม่นยำร้อยละ 100 แสดงให้เห็นว่าระบบการตัดสินใจเลือกช่องจอดทำงานได้อย่างถูกต้องและเชื่อถือได้ในทุกสถานการณ์ที่ทดสอบ

4.4 การทดสอบเซนเซอร์ Ultrasonic ตรวจจับสถานะช่องจอด

การทดสอบในส่วนนี้มีวัตถุประสงค์เพื่อตรวจสอบความแม่นยำของเซนเซอร์ Ultrasonic ในการตรวจจับสถานะของช่องจอดตรงว่ามีรถจอดอยู่หรือไม่ โดยระบบใช้ค่าระยะทางที่วัดได้เป็นเกณฑ์ในการตัดสินใจ กล่าวคือ หากเซนเซอร์วัดระยะทางได้น้อยกว่า 15 เซนติเมตร ระบบจะแปลผลว่ามีรถจอดอยู่ในช่องนั้น และหากวัดระยะทางได้มากกว่า 15 เซนติเมตร ระบบจะแปลผลว่าช่องจอดว่างอยู่

วิธีการทดสอบ

ดำเนินการทดสอบโดยนำรถยนต์จำลองวางและถอดออกสลับกันในช่องจอดตั้งแต่ช่อง A1 ถึง A6 โดยทดสอบช่องละ 10 ครั้ง รวมทั้งสิ้น 60 ครั้ง บันทึกผลในแต่ละครั้งว่าระบบสามารถตรวจจับสถานะได้ถูกต้องหรือไม่ โดยแบ่งการตรวจสอบออกเป็น 2 สถานะ ได้แก่ สถานะมีรถ ซึ่งทดสอบโดยวางวัตถุจำลองในช่องจอดให้อยู่ในระยะน้อยกว่า 15 เซนติเมตรจากเซนเซอร์ และสถานะไม่มีรถ ซึ่งทดสอบโดยนำวัตถุจำลองออกจากช่องจอดเพื่อให้ระยะที่วัดได้มากกว่า 15 เซนติเมตร จากนั้นบันทึกว่าระบบแสดงผลสถานะตรงกับความเป็นจริงหรือไม่ในแต่ละครั้ง ซึ่งแสดงรายละเอียดดัง ตารางที่ 4.3

ตารางที่ 4.4 ผลการทดสอบ Ultrasonic ตรวจจับสถานะช่องจอด

ช่องจอด	ตรวจจับถูก	ตรวจจับผิด
A1	10	0
A2	9	1
A3	8	2
A4	10	0
A5	10	0
A6	10	0

ผลการทดสอบ

พบว่าจากการทดสอบทั้งหมด 60 ครั้ง ใน 6 ช่องจอด ช่องละ 10 ครั้ง ระบบสามารถตรวจจับสถานะได้ถูกต้อง 57 ครั้ง และตรวจจับผิดพลาด 3 ครั้ง คิดเป็นความแม่นยำโดยรวมร้อยละ 95

เมื่อพิจารณารายช่อง พบว่าช่อง A1 A4 A5 และ A6 สามารถตรวจจับสถานะได้ถูกต้องครบทั้ง 10 ครั้ง คิดเป็นความแม่นยำร้อยละ 100 ในขณะที่ช่อง A2 ตรวจจับถูกต้อง 9 ครั้ง และผิดพลาด 1 ครั้ง คิดเป็นความแม่นยำร้อยละ 90 และช่อง A3 ตรวจจับถูกต้อง 8 ครั้ง และผิดพลาด 2 ครั้ง คิดเป็นความแม่นยำร้อยละ 80 ซึ่งช่อง A3 มีความผิดพลาดสูงที่สุดในบรรดาทุกช่องจอด

ความผิดพลาดที่เกิดขึ้นในช่อง A2 และ A3 มีสาเหตุมาจากการที่วัตถุจำลองถูกวางเข้าใกล้เซนเซอร์มากเกินไป ส่งผลให้ค่าระยะทางที่วัดได้อยู่นอกช่วงการทำงานที่เหมาะสมของเซนเซอร์ Ultrasonic ซึ่งเมื่อวัตถุอยู่ในระยะที่ใกล้เกินไป เซนเซอร์อาจไม่สามารถรับสัญญาณสะท้อนกลับได้อย่างถูกต้อง ทำให้ระบบแปลผลสถานะผิดพลาดได้

4.5 การทดสอบ Buzzer เมื่อป้ายทะเบียน RPI 5 ไม่ตรงกับ Ticket ID

การทดสอบในส่วนนี้มีวัตถุประสงค์เพื่อตรวจสอบการทำงานของ Buzzer ในกรณีที่ป้ายทะเบียนที่ RPI 5 อ่านได้ในลานจอดไม่ตรงกับข้อมูล Ticket ID ที่บันทึกไว้ใน Firebase ณ เวลาที่รถเข้าทางเข้า ซึ่งอาจเกิดจากการที่รถจอดผิดช่องหรือป้ายทะเบียนที่อ่านได้ไม่ตรงกับที่ระบบจัดสรรไว้ โดยระบบได้กำหนดให้ Buzzer ส่งสัญญาณเตือนจำนวน 3 ครั้ง ในลักษณะบีบ บีบ บีบ ซึ่งนับเป็น 1 รอบสัญญาณ ผ่านฟังก์ชัน beep_err() เมื่อตรวจพบความไม่ตรงกันดังกล่าว

วิธีการทดสอบ

ดำเนินการทดสอบโดยจงใจนำป้ายทะเบียนที่ไม่ตรงกับ Ticket ID ที่ระบบจัดสรรไว้เข้าจอดในช่องจอดตั้งแต่ช่อง A1 ถึง A6 ทีละช่อง รวมทั้งสิ้น 6 ครั้ง โดยในแต่ละครั้งบันทึกผลใน 3 ประเด็น ได้แก่ ประเด็นแรกคือ Buzzer ส่งสัญญาณเตือนหรือไม่ ประเด็นที่สองคือ Buzzer ดังครบ 3 ครั้ง ในลักษณะบีบ บีบ บีบ ซึ่งนับเป็น 1 รอบสัญญาณเตือนหรือไม่ และประเด็นที่สามคือระบบดำเนินการอ่านป้ายทะเบียนซ้ำหรือไม่หลังจากตรวจพบความไม่ตรงกัน ทั้งนี้หากระบบไม่สามารถจับคู่ป้ายทะเบียนได้สำเร็จภายใน 8 ครั้ง ระบบจะแสดงสถานะ Error และหยุดดำเนินการอ่านช่องจอดนั้น ซึ่งแสดงรายละเอียดดัง ตารางที่ 4.3

ตารางที่ 4.5 ผลการทดสอบ Buzzer เมื่อป้ายทะเบียนไม่ตรง

ช่อง	ป้ายทะเบียน Ticket ID	ป้ายทะเบียน ที่ RPI 5 อ่าน ได้	Buzzer ส่งสัญญาณ เตือน	จำนวน Buzzer ดัง	อ่านป้ายทะเบียนซ้ำ
A1	7กจ 3572	7กจ 3572	ไม่ดัง	0	ไม่อ่าน
A2	กก 2563	กก 2563	ไม่ดัง	0	ไม่อ่าน
A3	วร 8765	วร 8765	ไม่ดัง	0	ไม่อ่าน

ตารางที่ 4.5 (ต่อ) ผลการทดสอบ Buzzer เมื่อป้ายทะเบียนไม่ตรง

ช่อง	ป้ายทะเบียน Ticket ID	ป้ายทะเบียน ที่ RPI 5 อ่าน ได้	Buzzer ส่งสัญญาณ เตือน	จำนวน Buzzer ดัง	อ่านป้ายทะเบียนซ้ำ
A4	กท 2058	กท 2058	ดัง	1	อ่าน
A5	ขย 6555	ขย 6555	ไม่ดัง	0	ไม่อ่าน
A6	รล 2015	รล 2015	ดัง	8	อ่าน

ผลการทดสอบ

พบว่าจากการทดสอบทั้งหมด 6 ช่อง ระบบทำงานได้ถูกต้องใน 4 ช่อง ได้แก่ ช่อง A1 A2 A3 และ A5 ซึ่งป้ายทะเบียนที่ RPI 5 อ่านได้ตรงกับ Ticket ID ทุกประการ Buzzer จึงไม่ส่งสัญญาณเตือนและระบบไม่ดำเนินการอ่านซ้ำ แสดงให้เห็นว่าระบบสามารถแยกแยะกรณีที่ป้ายทะเบียนตรงกันได้อย่างถูกต้อง อย่างไรก็ตาม พบความผิดปกติใน 2 ช่อง ได้แก่ ช่อง A4 และ A6 โดยมีรายละเอียดดังนี้

ในกรณีช่อง A4 ระบบตรวจพบความไม่ตรงกันของป้ายทะเบียน Buzzer ส่งสัญญาณเตือน 1 ครั้ง และระบบดำเนินการอ่านป้ายทะเบียนซ้ำ ซึ่งความผิดพลาดที่เกิดขึ้นมีสาเหตุมาจากการที่ระบบอ่านป้ายทะเบียนผิดพลาดในเบื้องต้น อันอาจเกิดจากสภาพแสงหรือคุณภาพของภาพที่ถ่ายได้ในขณะนั้นไม่เหมาะสม ทำให้ระบบตีความตัวอักษรหรือตัวเลขบนป้ายทะเบียนคลาดเคลื่อนไปจากความเป็นจริง

ในกรณีช่อง A6 ระบบตรวจพบความไม่ตรงกันและ Buzzer ส่งสัญญาณเตือนถึง 8 ครั้ง ซึ่งเป็นจำนวนสูงสุดที่กำหนดไว้ ส่งผลให้ระบบแสดงสถานะ Error และหยุดดำเนินการอ่านช่องจอดนั้น ความผิดพลาดในกรณีนี้มีสาเหตุมาจาก RPI 4 ส่งข้อมูลป้ายทะเบียนที่บริเวณทางเข้ามาผิดพลาดตั้งแต่ต้น ทำให้ข้อมูลที่บันทึกใน Firebase ไม่ตรงกับป้ายทะเบียนจริงของรถที่เข้ามาจอด ระบบจึงไม่สามารถจับคู่ได้สำเร็จแม้จะพยายามอ่านซ้ำครบ 8 ครั้งแล้ว กรณีนี้จำเป็นต้องให้ผู้ดูแลระบบ (Admin) เข้ามาตรวจสอบและแก้ไขข้อมูลใน Firebase ด้วยตนเอง

โดยภาพรวม ระบบ Buzzer ทำงานได้ถูกต้องตามเงื่อนไขที่กำหนดในทุกกรณี กล่าวคือส่งสัญญาณเตือนเมื่อตรวจพบความไม่ตรงกัน และไม่ส่งสัญญาณเมื่อข้อมูลถูกต้อง อย่างไรก็ตามความผิดพลาดที่เกิดขึ้นชี้ให้เห็นว่าความแม่นยำของระบบโดยรวมยังขึ้นอยู่กับคุณภาพการอ่านป้ายทะเบียนของทั้ง RPI 4 และ RPI 5 ซึ่งควรได้รับการพัฒนาต่อไปเพื่อลดโอกาสเกิดข้อผิดพลาดดังกล่าว

สรุปผลการทดสอบทั้งหมด

จากการทดสอบระบบทั้งหมด 5 การทดสอบ สามารถสรุปผลได้ดังนี้

การทดสอบที่ 4.1 การอ่านป้ายทะเบียนทางเข้า พบว่าระบบสามารถอ่านป้ายทะเบียนได้ถูกต้อง 17 ครั้ง จากทั้งหมด 18 ครั้ง คิดเป็นความแม่นยำร้อยละ 94.44 โดยมีข้อผิดพลาดเพียง 1 ครั้ง ซึ่งเกิดจากสภาพแสงและคุณภาพของภาพที่ไม่เหมาะสมในขณะนั้น และระบบสามารถแก้ไขได้เองในรอบถัดไป

การทดสอบที่ 4.2 การเปรียบเทียบป้ายทะเบียนระหว่างทางเข้าและลานจอด พบว่าระบบสามารถจับคู่ป้ายทะเบียนได้สำเร็จครบทั้ง 6 คัน คิดเป็นอัตราความสำเร็จร้อยละ 100 โดยใช้จำนวนรอบเฉลี่ย 2 รอบต่อคัน และทุกคันอยู่ภายในเกณฑ์ที่กำหนดไว้ไม่เกิน 8 รอบ

การทดสอบที่ 4.3 การตัดสินใจเลือกช่องจอด พบว่าระบบสามารถเลือกช่องจอดได้ถูกต้องตามลำดับความสำคัญ A1 ถึง A6 ครบทุกกรณีทดสอบ ทั้งในกรณีที่ช่องว่างทุกช่อง ช่องคู่ว่าง และช่องคี่ว่าง คิดเป็นความแม่นยำร้อยละ 100

การทดสอบที่ 4.4 เซนเซอร์ Ultrasonic ตรวจจับสถานะช่องจอด พบว่าระบบตรวจจับสถานะได้ถูกต้อง 57 ครั้ง จากทั้งหมด 60 ครั้ง คิดเป็นความแม่นยำร้อยละ 95 โดยความผิดพลาดที่เกิดขึ้นในช่อง A2 และ A3 มีสาเหตุมาจากการวางวัตถุจำลองเข้าใกล้เซนเซอร์มากเกินไป ซึ่งในสภาวะการใช้งานจริงมีโอกาสเกิดขึ้นน้อย

การทดสอบที่ 4.5 การทำงานของ Buzzer เมื่อป้ายทะเบียนไม่ตรงกัน พบว่าระบบ Buzzer ทำงานได้ถูกต้องตามเงื่อนไขที่กำหนดในทุกกรณี โดยส่งสัญญาณเตือนเมื่อตรวจพบความไม่ตรงกัน และไม่ส่งสัญญาณเมื่อข้อมูลถูกต้อง อย่างไรก็ตามพบปัญหาในช่อง A4 ที่เกิดจากคุณภาพภาพในการอ่านป้ายและช่อง A6 ที่เกิดจากความผิดพลาดในการส่งข้อมูลของ RPI 4 ซึ่งกรณีหลังจำเป็นต้องให้ผู้ดูแลระบบเข้ามาแก้ไขด้วยตนเอง

โดยภาพรวม ระบบสามารถทำงานได้อย่างมีประสิทธิภาพในระดับที่ยอมรับได้ โดยการทดสอบส่วนใหญ่มีความแม่นยำสูงกว่าร้อยละ 90 ข้อผิดพลาดที่พบส่วนใหญ่มีสาเหตุมาจากปัจจัยภายนอก เช่น สภาพแสง คุณภาพของภาพ และระยะการติดตั้งเซนเซอร์ มิใช่ข้อบกพร่องของระบบโดยตรง ซึ่งสามารถแก้ไขและพัฒนาต่อไปในอนาคต

บทที่ 5

สรุปผลการทดลองและข้อเสนอแนะ

5.1 สรุปผลการทดลอง

โครงการนี้พัฒนาระบบผู้ช่วยจัดการพื้นที่จอดรถอัตโนมัติ (APMAS) โดยใช้การประมวลผลภาพร่วมกับ RPI 4 และ RPI 5 เพื่ออ่านและเปรียบเทียบป้ายทะเบียนรถยนต์ พร้อมทั้งจัดสรรช่องจอดโดยอัตโนมัติและตรวจสอบสถานะช่องจอดผ่านเซนเซอร์ Ultrasonic โดยบันทึกและอัปเดตข้อมูลแบบเรียลไทม์ผ่าน Firebase จากผลการทดสอบทั้ง 5 การทดสอบ พบว่าระบบสามารถทำงานได้อย่างมีประสิทธิภาพในระดับที่ยอมรับได้ โดยการอ่านป้ายทะเบียนมีความแม่นยำร้อยละ 94.44 การเปรียบเทียบและจับคู่ป้ายทะเบียนสำเร็จร้อยละ 100 การเลือกช่องจอดตามลำดับความสำคัญถูกต้องร้อยละ 100 การตรวจจับสถานะช่องจอดด้วยเซนเซอร์ Ultrasonic มีความแม่นยำร้อยละ 95 และระบบ Buzzer ส่งสัญญาณเตือนได้ถูกต้องตามเงื่อนไขที่กำหนดทุกกรณี

5.2 ปัญหาที่พบ

1. ระบบการอ่านป้ายทะเบียนมีความผิดพลาดในบางครั้ง เนื่องจากสภาพแสงและคุณภาพของภาพที่ถ่ายได้ไม่เหมาะสม ส่งผลให้ระบบอ่านตัวอักษรหรือตัวเลขบนป้ายทะเบียนคลาดเคลื่อน และต้องดำเนินการอ่านซ้ำหลายรอบ
2. Ultrasonic เกิดความผิดพลาดในการตรวจจับสถานะช่องจอด เมื่อวัตถุหรือรถยนต์อยู่ในระยะที่ใกล้เซนเซอร์มากเกินไป ทำให้เซนเซอร์ไม่สามารถรับสัญญาณสะท้อนกลับได้อย่างถูกต้อง
3. เมื่อ RPI 4 ส่งข้อมูลป้ายทะเบียนผิดพลาดมายัง Firebase ระบบไม่สามารถแก้ไขได้โดยอัตโนมัติ จำเป็นต้องให้ผู้ดูแลระบบเข้ามาตรวจสอบและแก้ไขข้อมูลด้วยตนเองทุกครั้ง

5.3 ข้อเสนอแนะ

1. ควรปรับปรุงระบบการอ่านป้ายทะเบียนโดยติดตั้งแสงไฟเสริมบริเวณกล้องหรือปรับค่าการประมวลผลภาพให้รองรับสภาพแสงที่หลากหลาย เพื่อเพิ่มความแม่นยำในการอ่านป้ายทะเบียนและลดจำนวนรอบการอ่านซ้ำ
2. ควรกำหนดระยะติดตั้งเซนเซอร์ Ultrasonic ให้เหมาะสมและชัดเจน รวมถึงอาจพิจารณาปรับค่าเกณฑ์การตรวจจับหรือเพิ่มการกรองสัญญาณรบกวน เพื่อลดโอกาสเกิดความผิดพลาดในการตรวจจับสถานะช่องจอด
3. ควรพัฒนาระบบตรวจสอบความถูกต้องของข้อมูลป้ายทะเบียนก่อนบันทึกลง Firebase และเพิ่มฟังก์ชันแจ้งเตือนผู้ดูแลระบบโดยอัตโนมัติเมื่อพบความผิดปกติ เพื่อลดการพึ่งพาการแก้ไขด้วยตนเองและเพิ่มความต่อเนื่องในการทำงานของระบบ

เอกสารอ้างอิง

- นาวาเอก ดร. ชัชวรินทร์ เลิศโยสธอร์ดิน. (2568). *Innovedex lesson*. ปัญญาประดิษฐ์ (Artificial Intelligence: AI) กำลังเปลี่ยนแปลงทุกอุตสาหกรรม การทำให้หุ่นยนต์มองเห็นและตัดสินใจตอบสนองทางกายภาพกลายเป็นทักษะพื้นฐานที่นักพัฒนายุคใหม่ต้องมี. สืบค้นจาก <https://www.chacharin.info/en/lessons/innovedex2025>
- ภาณุวัฒน์ คำจันทร์, ฉันทิตา สกลโอฬาร, และปวีรา แปะอุด. (2566). ระบบลานจอดรถอัจฉริยะ (Smart Parking) จำลองการทำงานที่จอดรถอัตโนมัติโดยใช้บอร์ดไมโครคอนโทรลเลอร์ Arduino UNO ในการควบคุมการทำงาน. สืบค้นจาก <https://princess-it.org/wp-content/uploads/2023/11/โครงการระบบลานจอดรถอัจฉริยะ-Smart-Parkingl-รร.รปค.56-sing-Sujan.pdf>
- ภาณุวัตร อุทัยบาล, และวนิดา แก่นอากาศ. (2566). วิเคราะห์พฤติกรรมในการจอดรถด้วยระบบจอดรถอัจฉริยะโดยใช้เทคโนโลยีอินเทอร์เน็ตทุกสรรพสิ่ง. สืบค้นจาก <https://ph01.tci-thaijo.org/index.php/fit-ssru/article/view/250626>
- วรุฒ พลอยเลี้ยง. (2566). การตรวจจับตัวอักษรป้ายทะเบียนรถยนต์ด้วยการเรียนรู้เชิงลึกเพื่อความปลอดภัยของผู้อยู่อาศัย. สืบค้นจาก <https://libdoc.dpu.ac.th/thesis/Warut.Ploi.pdf>
- วิกิพีเดีย สารานุกรมเสรี. (2569, 26 กุมภาพันธ์). *Raspberry Pi*. สืบค้นจาก https://en.wikipedia.org/wiki/Raspberry_Pi
- อลงกรณ์ พรสุขศิริ. (2565). แบบจำลองระบบจอดรถอัตโนมัติเพื่อประเมินประสิทธิภาพของระบบ ช่วยลดปัญหาเกี่ยวกับที่จอดรถ และช่วยให้การลงทุนด้านอสังหาริมทรัพย์. สืบค้นจาก https://digital.car.chula.ac.th/cgi/viewcontent.cgi?params=/context/chulaetd/article/7614/&path_info=6470305521.pdf